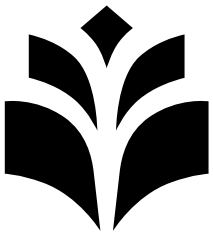
Luonnollisen kielen käsittely ja kysymyksiin vastaaminen usein kysytyissä kysymyksissä

Simo Turunen

Pro gradu -tutkielma



UNIVERSITY OF
EASTERN FINLAND

Tietojenkäsittelytieteen laitos
Tietojenkäsittelytiede
Joulukuu 2023

ITÄ-SUOMEN YLIOPISTO, Luonnontieteiden, metsätieteiden ja tekniikan tiedekunta
Joensuu
Tietojenkäsittelytieteen laitos
Tietojenkäsittelytiede

Turunen, Simo: Luonnollisen kielen käsittely ja kysymyksiin vastaaminen usein kysytyissä kysymyksissä
Pro gradu -tutkielma, 62 s.
Ohjaajat: Pauli Miettinen ja Henri Leisma
Joulukuu 2023

Tiivistelmä: Tutkielmassa esitetään kuvaus luonnollisen kielen käsittelytekniikoihin, joita käytetään kysymyksiin vastaamiseen tarkoitettujen järjestelmien kehittämisessä. Internetissä saatavilla olevan tiedon määrän kasvaessa räjähdysmäisesti, on tullut yhä haastavammaksi käsitellä suuria tietomääriä sekä vastata verkkopalveluiden käyttäjien tarpeisiin. Usein kysytyt kysymykset (FAQ) -sivut ovat olleet yleinen ratkaisu, mutta ne eivät useinkaan pysty vastaamaan erityisiin ja vivahteikkaisiin kyselyihin. Ongelmia esiintyy esimerkiksi manuaalisessa sivun päivittämisessä, sekä rajoitteissa ennalta määrättyihin kysymyksiin ja vastauksiin. Tässä yhteydessä luonnollisen kielen käsittelyn menetelmät tarjoavat lupaavan väylän kysymysten vastausjärjestelmien parantamiseen.

Luonnollisen kielen käsittelyn keinot tekstin esikäsittelystä, luonnollisen kielen analyysin keinoin sekä tiedonhaun avulla mahdollistavat klassisten kysymyksiinvastaamisjärjestelmien luonnin. Menetelmien avaaminen auttaa hahmottamaan kysymysten vastausjärjestelmiä, jotka lopulta tarjoavat mahdollisuuden vastata automaattisesti usein kysyttyihin kysymyksiin. Alan dynaamisen luonteen vuoksi luonnollisen kielen käsittelyn yleisesti hyväksytty määritelmä on vaikeasti löydettävissä.

Tutkielmasta käy ilmi alan monitieteinen luonne sekä yhteistyö kielitieteen, tietojenkäsittelytieteen ja kognitiivisen psykologian välillä. Nykytilannetta luonnollisen kielen käsittelyssä sivutaan esitettyjen kysymysten vastausjärjestelmien avulla. Pääpaino pysyy kuitenkin klassisiksi todetuilla menetelmillä, joita käytetään edelleen ja ovat luonnollisen kielen käsittelyn perustaa. Lisäksi tutkielmassa käsitellään suomen kieltä, jota pidetään morfologisesti rikkaana kielenä, ja sen morfologinen rikkaus tekee haastavammaksi kehittää kysymyksiinvastaamisjärjestelmiä.

Avainsanat: Luonnollisen kielen käsittely; Luonnollisen kielen ymmärtäminen; Luonnollisen kielen generointi; NLP; NLU; NLG; Kysymyksiin vastaaminen;

ACM CCS (2012)
• **Computing methodologies** → **Natural language processing**; *Artificial intelligence*;

UNIVERSITY OF EASTERN FINLAND, Faculty of Science, Forestry and Technology
Joensuu
School of Computing
Computer Science

Turunen, Simo: Natural language processing and question answering in frequently asked questions
Master's thesis, 62 p.
Supervisors: Pauli Miettinen and Henri Leisma
December 2023

Abstract: The thesis presents a description of natural language processing techniques used in the development of question answering systems. As the amount of information available on the Internet has grown exponentially, it has become increasingly challenging to process large amounts of data and to meet the needs of users of online services. Frequently asked questions (FAQ) pages have been a common solution, but they are often unable to respond to specific and nuanced queries. Problems include manual updating of the page, and limitations on predefined questions and answers. In this context, natural language processing methods offer a promising avenue for improving question answering systems.

Natural language processing methods from text preprocessing, natural language analysis, and information retrieval allow the creation of classical question answering systems. Unlocking these methods will help to outline question answering systems that will eventually provide the possibility of automatically answering frequently asked questions. Due to the dynamic nature of the field, a generally accepted definition of natural language processing is difficult to find.

The thesis reflects the interdisciplinary nature of the field and the collaboration between linguistics, computer science and cognitive psychology. The current state of the art in natural language processing is addressed through the question answering systems presented. However, the emphasis will remain on methods that have been proven to be classical and are still used and form the basis of natural language processing. Furthermore, the thesis deals with Finnish, considered as a morphologically rich language, and its morphological richness makes it more challenging to develop question answering systems.

Keywords: Natural Language Processing; Natural Language Understanding; Natural Language Generation; NLP; NLU; NLG; Question-Answering;

ACM CCS (2012)
• **Computing methodologies** → **Natural language processing**.*Artificial intelligence*;

Sisällys

1 Johdanto **1**

1.1 Tutkimuksen tavoite ja tutkimuskysymykset 2

1.2 Luonnollisen kielen käsittelyn historia 3

2 Luonnollisen kielen käsittely **6**

2.1 Esikäsittely 7

2.1.1 Stemmaus 9

2.1.2 Lemmatisointi 11

2.2 Luonnollisen kielen analyysikeinot 12

2.3 Lähestymistavat luonnollisen kielen oppimiseen 15

2.4 Säännölliset lausekkeet 16

2.5 Käyttökohteet 18

3 Kysymyksiin vastaamisen menetelmät **20**

3.1 Tiedonhakujärjestelmät 21

3.1.1 Tiedonhakumallit 23

3.2 Sanaluokkien tunnistaminen 26

3.3 Nimettyjen entiteettien tunnistaminen 27

3.4 Entiteettien yhdistäminen 30

3.5 Tietopohjaiset kysymyksiinvastaamisjärjestelmät 31

4 Kysymyksiinvastaamisjärjestelmät **33**

4.1 Kysymysten käsittely 35

4.2 Asiakirjojen käsittely 36

4.3 Vastauksen käsittely 36

4.4 Vastauksen tuottaminen 37

4.4.1 Prosessijakauma 37

4.5 Klassiset kysymyksiin vastaamisen mallit 39

4.6 Rasa–vuoropuhelun ja luonnollisen kielen ymmärtämisen viitekehys . 41

4.7 TextRunner-tiedonpoimintajärjestelmä 43

5 Suomen kielen sovittaminen luonnollisen kielen käsittelyyn **48**

5.1 Suomen kielen haasteet ja ongelmat 48

5.2 Edistyksiä luonnollisen kielen käsittelyn ratkaisuissa 50

6 Yhteenveto **53**

6.1 Kysymyksiin vastaaminen usein kysytyissä kysymyksissä 55

6.2 Suositukset ja tulevaisuuden suuntaviivat 57

Viitteet **58**

1. Johdanto

Internetin tiedon määrän kasvaessa navigoiminen lähteeseen, joka tarjoaa suoran vastauksen kysymykseen, muuttuu yhä hankalammaksi. Verkkopalvelut ovat alkaneet sisältää valtavia tietomääriä, ja palvelua tuntematon käyttäjä ei löydä tarvitsemaansa tietoa helposti. Kyseiseen ongelmaan tarjotaan ratkaisuna usein kysyttyjen kysymysten sivua, joka tarjoaa yleisiin asioihin tukea. Kuitenkin tämä on rajallinen tapa auttaa loppukäyttäjää saavuttamaan tietoa, sillä hänellä voi olla tiedonpuutetta aiheesta, joka ei ole päässyt usein kysyttyjen kysymysten sivulle. Jijkoun ja de Rijke (2005) mainitsevat usein kysyttyjen kysymysten sivujen olevan tärkeä resurssi kohdennettuun tiedontarpeeseen. He lisäävät, että noin 36 prosenttia näistä sivuista vastasi käyttäjien tarpeeseen. Prosentuaalisesti kasvunvaraa on vielä huomattava määrä, ja kysymyksiinvastaamisjärjestelmiä voidaan pitää potentiaalisena prosenttimäärän kasvattajana. Kysymyksiinvastaamisjärjestelmän esiintyminen verkkosivulla voi esiintyä monessa muodossa, kuten tekstikenttänä, johon käyttäjät voivat syöttää kysymyksiä, dialogimaisena keskustelubottina tai järjestelmänä, joka voi louhia kontekstiin sopivat kysymykset ja tarjota ne automaattisesti listana. Nämä kysymyksiinvastaamisjärjestelmät rakentuvat erilaisista luonnollisen kielen käsittelyn metodeista, joita käsitellään tarkemmin tutkielman aikana.

Jackson ja Moulinier (2002) kertovat luonnollisen kielen käsittelyn (engl. *natural language processing*) kuvastavan keinoja, joilla tietokoneet voivat analysoida tai tuottaa kirjoitusta ja puhetta. He selventävät myös, että luonnollisen kielen käsittelyä pidetään yläkategoriana luonnollisen kielen ymmärtämiselle (engl. *natural language understanding*) ja luonnollisen kielen tuottamiselle (engl. *natural language generation*). Luonnollinen kieli on kehittynyt ihmisten väliseen kommunikaatioon (Tieteen termipankki, 2023). Vastakohtana luonnolliselle kielelle ovat keinotekoiset kielet, jotka ovat luotu tarkoin määriteltyä tarkoitusta varten. Esimerkiksi ohjelmointikielet ja elokuvissa esiintyvät keksityt kielet ovat keinotekoisia kieliä. Liddy (2001) kuvailee luonnollisen kielen käsittelyä automaattisena tekstin analysointimenetelmänä, joka sisältää sekä teoreettisia periaatteita että teknisiä välineitä. Hän huomioi luonnollisen kielen käsittelyn olevan keino saavuttaa päämäärä, eikä valmis työkalu käsittelemään luonnollista kieltä. Yleisenä

luonnollisen kielen käsittelyn päämääränä on pidetty luonnollisen kielen ymmärtämistä. Luonnollisen kielen ymmärtäminen on haasteellinen päämäärä, eikä sitä ole vieläkään täysin saavutettu. Täydellinen luonnollisen kielen ymmärtämisjärjestelmä pystyisi kertomaan omin sanoin annetun tekstin sisällön, kääntämään tekstin toiselle kielelle, vastaamaan tekstissä ilmeneviin kysymyksiin, sekä tekemään johtopäätöksiä tekstistä. Tämä tutkielma käsittelee luonnollisen kielen käsittelyä usein kysyttyjen kysymyksien vastaamisen tukena, joka on yksi täydellisen luonnollisen kielen ymmärtämisjärjestelmän vaatimuksista. Luonnollisen kielen käsittelyn tieteenalaa pidetään dynaamisena ja kehittyvänä, jonka vuoksi tarkan määritelmän muodostaminen on hankalaa (Liddy, 2001). Liddyn (2001) mukaan tieteenryhmät korostavat eri näkökohtia luonnollisen kielen käsittelystä, jotka perustuvat niiden tieteelliseen alkuperään.

Liddy (2001) kertoo, että luonnollisen kielen käsittelystä käytettiin nimitystä laskennallinen kielitiede (engl. *computational linguistics*). Merkitykset ovat kuitenkin jakaneet termit erilleen ja laskennallisen kielitieteen ja luonnollisen kielen käsittelyllä on ero. Liddyn (2001) mukaan luonnollisen kielen käsittelyn tärkeimpiin tekijöihin kuuluu kielitiede (engl. *linguistics*), tietojenkäsittelytiede (engl. *computer science*) sekä kognitiivinen psykologia (engl. *cognitive psychology*). Laskennallinen kielitiede on tutkimusala, joka keskittyy muodollisiin ja rakenteellisiin kielimalleihin sekä kielten universaalien piirteiden löytämiseen (Liddy, 2001). Tietojenkäsittelytiede keskittyy sisäisten tietorepresentaatioiden ja tehokkaiden käsittelymenetelmien kehittämiseen (Liddy, 2001). Kognitiivinen psykologia tutkii kielenkäyttöä keinona ymmärtää ihmisen kognitiivisia prosesseja, ja pyrkii luomaan psykologisesti uskottavia kielenkäyttömalleja (Liddy, 2001). Usea tieteenala tuo oman osuutensa luonnollisen kielen käsittelyyn. Seuraavana tarkastellaan varhaisimpia luonnollisen kielen käsittelyn mahdollistamia sovelluksia.

1.1 Tutkimuksen tavoite ja tutkimuskysymykset

Tutkimuksen tavoitteena on tutkia mistä klassiset kysymyksiinvastaamisjärjestelmät ovat koostuneet. Kontekstina pysyy verkkopalveluiden usein kysyttyjen kysymysten sivu, joka heijastuu tutkimuksessa läpikäytäviin asioihin. Läpikäytävät esimerkit muodostetaan englannin kielelle, sillä suomen kielen vaikutuksia käydään omassa kappaleessa läpi.

Tutkimus pyrkii tuomaan selvän kokonaisuuden mitä kysymyksiinvastaamisjärjestelmissä pitää ottaa huomioon. Parhaimmillaan tutkimus voi auttaa luomaan tietopohjan, jota voidaan käyttää hyväksi kehittämään usein kysyttyjen kysymysten sivua. Parannuskohteet ovat erityisesti tarpeellisia, kun laajan verkkopalvelun tiedot ovat hajautuneet ja tiedon löydettävyyttä sekä liitettävyyttä halutaan parantaa.

En ota tutkimuksessani huomioon esimerkiksi nykypäivänä suosiossa olevia chatbotteja, kuten ChatGPT:ta. Useat näistä eivät noudata avoimen lähdekoodin periaatteita, ja niistä analysoitavissa oleva materiaali on minimaalista.

Tutkimuskysymykset ovat seuraavat:

1. Miten luonnollisen kielen käsittelyä voidaan hyödyntää antamaan vastauksia verkkopalveluiden usein kysyttyjen kysymysten kontekstissa?
2. Mitä ongelmakohtia suomen kieli tuo luonnollisen kielen käsittelyn metodeihin?

Ensimmäisen tutkimuskysymyksen vastaamisen tueksi tutkielmassa esitellään lyhyesti luonnollisen kielen käsittelyn historiaa. Tämän jälkeen käsitellään yleisiä luonnollisen kielen käsittelyn metodeja sekä niiden käyttökohteita. Kun nämä aiheet on käyty läpi, voidaan esitellä kysymyksiin vastaamisessa käytettäviä spesifejä metodeja ja aihepiirejä. Lisäksi näiden jälkeen käydään läpi erilaisia kysymyksiinvastaamisjärjestelmiä, jotta luvussa 6 sekä sen aliluvussa 6.1 voidaan arvioida ensimmäistä tutkimuskysymystä. Toinen tutkimuskysymys tulee esiin luvussa 5, joka esittelee suomen kielen tilannetta luonnollisen kielen käsittelyssä.

1.2 Luonnollisen kielen käsittelyn historia

Liddy (2001) mainitsee luonnollisen kielen käsittelyn tutkimuksen saaneen alkunsa jo 1940-luvun lopulla. Hän kertoo konekääntämisen olleen varhaisin luonnollisen kielen käsittelyn sovellus, ja että ensimmäiset konekääntämisen hankkeet saivat alkunsa toisen maailmansodan aikaisesta koodinmurto-osaamisesta. Varhaiset konekääntämisen järjestelmät keskittyivät pääasiassa sanastoon ja sanajärjestykseen, mikä johti käännösten huonoon laatuun, koska muun muassa sanojen moniselitteisyys jätettiin huomiotta (Liddy, 2001). Martin (2023) kertoo samanaikaisesti 1940-luvulla olleen kehitystä neuroverkkojen parissa, kun McCulloch ja Pitts mallinsivat ihmisen kaltaista neuronia esittämällä Boolen logiikkaa. Chomskyn vuonna 1957 esittämä generatiivinen kielioppi tarjosi tietoa kieliteorian mahdollisuuksista konekääntämisessä (Liddy, 2001). Liddy (2001) mainitsee, että kyseisenä ajankohtana luonnollisten kielten sovellukset laajenivat kattamaan myös puheentunnistusta, mikä lopulta johti jakoon teoreettisen kielitieteen ja tilastollisten menetelmien välillä.

Liddy (2001) noteeraa, että 1950-luvulla uskottiin, että täysin automaattiset ja korkealaatuiset käännösjärjestelmät olisivat toteutettavissa muutaman vuoden sisällä. Hän myös

kertoo, että tämä osoittautui pian epärealistiseksi. Martin (2023) mainitsee neuroverkkojen saaneen kehitykseen vauhtia, jonka mukana kehittyi neuroverkkojen perustana tunnettu perceptron. Vuonna 1966 laadittu ALPAC-raporttissa (Automatic Language Processing Advisory Committee of the National Academy of Science) suositeltiin konekääntämisen rahoituksen lopettamista (Liddy, 2001). Liddyn (2001) mukaan kyseinen raportti oli askel luonnollisen kielen käsittelyn tutkimuksen hidastumiseen Yhdysvalloissa. Hän mainitsee, että tutkimuksia käsittelevien prototyyppijärjestelmien rakentaminen sekä teoreettinen kehitys kuitenkin jatkui vaikeuksista huolimatta. 1960- ja 1970-lukujen kehitys keskittyi selvittämään kuinka esittää sanojen ja lauseiden merkitystä, sekä kuinka ratkaistaan aikaisempien vuosien kielioppiongelmat laskennallisesti tehokkaasti (Liddy, 2001).

Liddy (2001) mainitsee aikaisempien prototyyppijärjestelmien, kuten Weizenbaumin ELIZA, Winogradin SHRDLU ja Colbyn PARRY, esittelevän luonnollisen kielen ymmärtämisominaisuuksia. Hän kertoo esimerkiksi, että ELIZA rakennettiin imitoimaan keskustelua potilaan ja psykologin välillä. Hänen mukaansa imitointi tapahtui yksinkertaisilla käyttäjän syötteen muunteluilla sekä toistolla. SHRDLU prototyyppijärjestelmä toisaalta käytti käyttäjän syötettä muokkaamaan pöydällä sijaitsevia puupalikkojen sijaintia käyttäen robottikättä (Liddy, 2001). Liddy (2001) noteeraa, että SHRDLU:n antama esimerkki antoi toivoa, että tietokoneet voisivat saavuttaa luonnollisen kielen ymmärtämisen ominaisuuksia. Hän kertoo myös, että PARRY-järjestelmä yritti esittää vainoharhaista ihmistä, ja teki niin käyttämällä avainsanojen joukkoa aktivoimaan seuraavaan vastauksen. Jos avainsanaa ei löytynyt suoraan, niin järjestelmä käytti synonyymikirjastoa apuna linkittämään syötteen mahdollisiin vastauksiin (Liddy, 2001). Liddyn (2001) mukaan 1970-luvun lopulla huomio siirtyi semanttisiin kysymyksiin ja diskurssi-ilmiöihin. Diskurssi kuvaa vuorovaikutuksen riippuvuuksia yhteiskunnallisiin normeihin ja valtasuhteisiin (Tieteen termipankki, 2021). Tutkijat analysoivat tehtäväkeskeisiä dialogeja ja esittivät diskurssin rakennetta koskevia teorioita (Liddy, 2001).

Liddy (2001) kertoo, että luonnollisen kielen tuottamiseen kiinnitettiin huomiota, ja McKeown TEXT:n kaltaiset järjestelmät tuottivat yhtenäisiä vastauksia retoristen predikaattien avulla. Retoriset predikaatit ovat merkkejä, jotka auttavat jäsentämään tuotettavaa tekstiä, tekstin välisten suhteiden avulla (Marcu, 1997). Esimerkiksi ihminen voidaan esitellä geolokaatiolla, luonteella ja fyysisillä ominaisuuksilla. Liddy (2001) kuvaa TEXT:n tarkoituksen olleen tuottaa yhtenäisiä ja johdonmukaisia tekstejä käyttämällä erilaisia diskurssianalyysin tekniikoita. Hän kuvaa myös 1980-luvun alussa tutkijoiden olleen jälleen kiinnostuneita ei-symbolisista lähestymistavoista. Ei-symboliset tavat eivät tukeudu vahvasti symbolisiin esityksiin, eli esimerkiksi avainsanoihin tai sääntöihin (Liddy, 2001). Kyseisiä lähestymistapoja ovat muun muassa tilastoihin perustuvat. Liddy

(2001) kuvaa ei-symbolisten lähestymistapojen kiinnostuksen johtuvan laskennallisesta kehityksestä ja yksittäisten ratkaisujen rajoitusten havaitsemisesta. Hän mainitsee myös, että 1980-luvun lopulla luonnollisen kielen käsittelyssä yleistyi symbolisten ja tilastollisten lähestymistapojen yhdistelmä, jonka oletettiin ratkaisevan tehokkaasti erilaisia ongelmia.

Liddy (2001) huomioi luonnollisen kielen käsittelyn alan kasvaneen nopeasti 1990-luvulla, mitä vauhdittivat suurten sähköisten tekstikokoelmien saatavuus, nopeammat tietokoneet ja Internetin laajeneminen. Hän kertoo tilastollisten lähestymistapojen nousseen esiin, ja että ne käsittelevät tehtäviä kuten puheenosien tunnistaminen ja sanojen merkitysten erottelu. Vuoteen 2021 mennessä asiantuntijat ovat kehittäneet seuraavan sukupolven luonnollisen kielen käsittelyn järjestelmiä, jotka pystyvät käsittelemään yleistä tekstiä ja ottamaan huomioon kielen vaihtelevuuden ja moniselitteisyyden merkittävässä määrin (Liddy, 2001). Martinin (2023) mukaan neuroverkot ovat myös kokeneet erilaisia suosion ja laskun vaiheita. Hänen mukaansa niiden käyttö lisääntyi 1980-luvulla erityisesti vastavirta-algoritmin (engl. *backpropagation*) käyttöönoton myötä. Lisäksi hän kertoo, että 2010-luku oli kuitenkin merkittävä käännekohta neuroverkoille, erityisesti syväoppimisen myötä. Suurten tietokokonaisuuksien ja tehokkaiden grafiikkasuorittimien saatavuus helpotti syvempien verkkojen kouluttamista, mikä johti läpimurtoihin erilaisissa luonnollisen kielen käsittelyn tehtävissä. Syväoppimisen mallien kehittymisen vuoksi rekursiiviset neuroverkot (engl. *recurrent neural networks*) sekä muuntajat (engl. *transformers*), muodostavat nykyisin huipputason sovelluksia (Martin, 2023, luku 7).

2. Luonnollisen kielen käsittely

Kuten jo aiemmin luvussa 1 mainittiin, tekstimuotoisen tiedon saatavuus on laaja sekä hajanainen. Oikean tiedon saaminen oikeaan aikaan voi olla haastavaa, jolloin luonnollisen kielen käsittely voi tuoda tukea kyseiseen ongelmaan.

Jackson ja Moulinier (2002) huomioivat, että kielen sanat ja lauseet voivat olla monimerkityksellisiä, jolloin sanan tai lauseen todellinen merkitys ei käy ilmi. Esimerkiksi sana ”bat” voi englanniksi tarkoittaa lepakkoa tai pesäpallomailaa. Sanojen ja lauseiden moniselitteisyys on yksi luonnollisen kielen käsittelyn ongelmakohdista. Indurkhya ja Damerau (2010) mainitsevat luonnollisten kielien omaavan paljon epäselvyyksiä, jonka vuoksi on tärkeää ottaa ne huomioon luonnollisen kielen käsittelyn sovelluksia kehitettäessä. He mainitsevat, että epäselvyyksien vuoksi luonnollisen kielen analysointi on tärkeä osa luonnollisen kielen käsittelyä.

Luonnollisen kielen käsittelyyn käytettävä materiaali voi olla tekstiä tai puhetta, mutta lopulta käsittely tehdään usein kuitenkin vain tekstimuotoon muutettuna (Jackson & Moulinier, 2002). Tässä tutkielmassa oletetaan materiaalin olevan pääosin tekstiä. Lounela (2006) määrittelee tietokoneella käytettävän kielentutkimusta avustavan materiaalin korpukseksi. Hän lisää, että korpus sisältää sähköisessä muodossa olevia tekstejä sekä nauhoitteita. Luonnollisen kielen käsittelyssä korpus tunnetaan järjestelmällisesti koottuna kirjoitettujen tai puhuttujen näytteiden kokonaisuutena (Tieteen termipankki, 2015).

Martin (2023) mainitsee, että useat luonnollisen kielen käsittelyä hyödyntävät sovellukset luodaan vain englannin kielelle. Suomen kieltä luonnollisen kielen käsittelyssä tutkitaan myöhemmässä luvussa 5. Martin (2023) huomioi, että parhaimmassa tapauksessa sovellukset ovat käytettävissä useille eri kielille. Hän noteeraa kuitenkin, että tämä on haastava tehtävä, sillä kielien eroavaisuudet eivät mahdollista yleispätevää toimintatapaa.

Luonnollisen kielen käsittelyn alkuvaiheena voidaan pitää tiedon hankkimista ja sen esikäsittelyä, joita esitellään seuraavassa alaluvussa. Luonnollisen kielen analyysikeinot

puolestaan auttavat luomaan rakenteellisen esityksen esitettävästä aiheesta kuten kysymyksestä, sekä hankkimaan tarkemman kuvauksen aiheen tarkoitetusta merkityksestä. Seuraavaksi esitetään säännölliset lausekkeet konkreettisena esimerkkinä työkalusta, jota voidaan hyödyntää luonnollisen kielen käsittelyn sovelluksissa.

2.1 Esikäsittely

Tekstin esikäsittelyä varten teksti pitää ensin hankkia tekstinlouhintaprosessin menetelmin. Vijayarani, Ilamathi ja Nithya (2015) kertovat tekstin louhinnan (engl. *text mining*) olevan prosessi, jolla haetaan hyödyllistä tietoa rakenteettomista teksteistä. He kuvaavat tekstin louhinnan prosessia samaksi kuin tiedon louhinta, mutta eroavaisuutena tiedon louhinta keskittyy rakenteelliseen dataan. He lisäävät myös, että tekstin louhinta pystyy hankkimaan tietoa sekä rakenteettomasta että puolirakenteellisesta datasta. Esimerkkinä mahdollisista louhinnan sijainneista Vijayarani ja muut (2015) kuvaavat olevan sähköpostit, HTML-asiakirjat sekä tekstiasiakirjat. Vijayarani ja muut (2015) kertovat, että rakenteellinen data sijaitsee kiinnitetyssä sijainnissa, joka tallennetaan usein relaatiotietokantoihin sekä erilaisiin taulukoihin. He mainitsevat, että rakenteeton data on vastakohta rakenteelliselle datalle, ja sijaitsee muualla kuin tavanomaisissa relaatiotietokannoissa. Puolirakenteinen data ei ole heidän mukaansa täysin raa'assa muodossa ilmenevää eikä sijaitse tavanomaisissa tietokannoissa. Puolirakenteista dataa voi ilmetä esimerkiksi JSON-tiedostossa, kun avainten arvokentät sisältävät vapaamuotoista tekstiä. Tekstin louhintaa hyödynnetään tiedonhakujärjestelmissä, tiedonlouhinnassa ja yleisissä luonnollisen kielen käsittelyn sovelluksissa (Vijayarani ja muut, 2015).

Indurkhya ja Damerau (2010) jakavat esikäsittelyn kahteen vaiheeseen, asiakirjojen seulomiseen ja tekstin segmentointiin. He kuvaavat asiakirjojen seulomista prosessiksi, joka muuttaa sekalaiset digitaaliset tiedostot yhdenmukaisiksi asiakirjoiksi. Eroavaisuuksia tiedostojen välillä voi olla esimerkiksi merkkijonojen koodauksissa (engl. *encoding*) (Indurkhya & Damerau, 2010), kuten UTF-8 ja ASCII. Kyseinen tilanne on suotavaa huomioida, jotta merkkijonot eivät pääse korruptoitumaan, joka puolestaan heikentää luonnollisen kielen käsittelyn prosessia. Toisena esimerkkinä asiakirjat voivat olla keskenään eri kielisiä, esimerkiksi yhdessä asiakirjassa voi olla useita kieliä. Eri kielet ovat yhteydessä eri merkkijonokoodauksiin, mutta eivät ole aina täysin verrannollisia toisiinsa. Yksinkertaisissa luonnollisen kielen sovelluksissa on hyödyllistä keskittyä yhteen kieleen, jotta käsittelystä vastaava sovellus ei muutu liian monimutkaiseksi. Indurkhya ja Damerau (2010) kertovat, että ennen tekstin segmentointia asiakirjojen epäolennainen sisältö pitää karsia pois. Asiakirjat sisältävät usein ylimääräistä tietoa, jotka eivät liity luonnolliseen kieleen (Indurkhya & Damerau, 2010). Esimerkkinä HTML tuomat

jaottelut kuten ” text ”. Kun asiakirjat ovat yhdenmukaistettu ja niiden ylimääräinen sisältö on karsittu pois, on mahdollista jatkaa tekstin segmentointiin (Indurkhya & Damerau, 2010).

Indurkhya ja Damerau (2010) esittelevät tekstin segmentointia prosessina, jossa tekstikokonaisuus pilkotaan pienempiin merkityksellisiin osiin kuten lauseet ja kappaleet. He kertovat, että tekstin jakaminen omiin kokonaisuuksiinsa tuo omat haasteensa, sillä esimerkiksi päivämäärät ja välimerkit voivat rikkoa tavanomaisen ”.”-erottelijan käytön. Erottelija määrää kohdan, mistä kokonaisuus jaetaan osiin. Tietoa voidaan muotoilla epäjohdonmukaisesti ja eri tavoin, kuten päivämäärät voidaan esittää numeraalisessa muodossa tai vapaana tekstinä (Reno, 2015). Esimerkiksi suomen kielessä päivämäärät voivat esiintyä muodossa 21.04.2023, ja englannin kielessä toisaalta 04/21/2023.

Indurkhya ja Damerau (2010) mainitsevat, että tekstin segmentointiin kuuluu myös normalisointi. Normalisointi on esikäsittelyn keino, jonka tarkoituksena on yhdenmukaistaa ja yksinkertaistaa läpikäytävää materiaalia (Martin, 2023). Martin (2023) kertoo, että esimerkiksi sanassa ”processing” sen taivutusmuodot normalisoidaan yhdeksi sanaksi. Sanoja kutsutaan tekstinkäsittelyssä tokeneiksi (Indurkhya & Damerau, 2010). Tokenisointi puolestaan tarkoittaa sanojen erottelua annetussa korpuksessa (Martin, 2023). Esimerkiksi lauseet: ”Natural Language Processing is interesting” ja ”Natural Language Understanding is hard”. Tällöin esimerkkilauseiden sanojen esiintymät muutetaan yksittäisiin esiintymiin, jossa ensimmäinen lause tokenisoidaan muotoon ["Natural", "Language", "Processing", "is", "interesting"]. Tokeneita on yhteensä esimerkkilauseessa $N = 10$, joka määrittyy sanamäärän mukaan. Sanastoa toisaalta kuvataan notaatiolla $|V|$, johon kuuluu jokainen erillinen sana vain kerran (Martin, 2023), jolloin esimerkkilauseilla tyyppimäärä on kuusi. Silloin esimerkkilauseista muodostuva sanasto on seuraavanlainen: ["Natural", "Language", "Processing", "Understanding", "is", "hard"]. Martin (2023) mainitsee, että deterministiset algoritmit, jotka perustuvat säännöllisiin lausekkeisiin, ovat perinteisesti pidettyjä tavanomaisina ja nopeina tapoina tokenisoida korpuksia.

Moniosaiset sanat tuovat oman haasteen tokenisointiin, esimerkiksi kaupunkien nimet (Martin, 2023). Suomessa yhdyssanat eivät koskaan ole eroteltu sanojen välisellä välilyönnillä (Hyönä, Bertram & Pollatsek, 2004). Esimerkiksi Yhdysvalloissa sijaitseva ”Los Angeles” kirjoitetaan erikseen. Martin (2023) mainitsee, että moniosaisia sanoja, jotka menettävät tarkoituksen erillään, on tapana käsitellä usein yhtenä tokenina. Hän kertoo myös, että luonnollisen kielen käsittelyssä voi esiintyä tilanteita, jolloin käsittelystä vastaava algoritmi kohtaa tuntemattoman sanan. Tilanteeseen pitää reagoida tunnistamalla onko sana oikea vai sivuutetaanko sana täysin. Martin (2023) huomioi, että normalisointiin käytettävät valmiit algoritmit saattavat lisätä tokeneiden joukkoon

odottamattomia morfeemeja. Morfeemit ovat pienin tarkoituksen omaava sananosa (Martin, 2023), kuten englanninkielisten sanojen etu ja takaliitteet. Esimerkiksi sana ”preheat” koostuu kahdesta morfeemista, ”pre” ja ”heat”. Morfeemit jaotellaan kahteen luokkaan, jotka ovat kantasanat ja liitteet (Martin, 2023). Heat kuvastaa kantasanaa ja sanan etuliite muokkaa kantasanan merkitystä, antaen sanalle lisämerkitystä. Martin (2023) kertoo, että morfeemien tutkimusta kutsutaan morfologiaksi.

2.1.1 Stemmaus

Eräs normalisointiin käytettävä algoritmityyppi on stemmaus. Martin (2023) kertoo, että stemmaus pyrkii sanojen perusmuodon selvittämiseen. Vijayarani ja muut (2015) huomioivat, että stemmauksen hyötynä on muun muassa käsiteltävän sanamäärän väheneminen, joka johtaa tila- ja aikavaativuuden pienenemiseen. He mainitsevat, että stemmauksen prosessi mahdollistaa tehokkaamman tekstinkäsittelyn ja -haun käsittelemällä toisiinsa liittyviä sanoja yhtenä kokonaisuutena. Lisäksi he erottelevat stemmaajat kolmeen eri kategoriaan: katkaisumenetelmiin, tilastollisiin menetelmiin, sekä hybridimenetelmiin.

Martin (2023) mainitsee tunnetun Porterin stemmaajan perustuvan sääntöihin, joita noudattamalla sanoista karsitaan takaliitteitä. Näin voidaan olettaa kyseisen stemmaajan kuuluvan katkaisumenetelmien kategoriaan. Voidaan esimerkiksi esittää sääntö, jossa englanninkielisten sanojen ”-ing” päätteiset liitteet poistetaan, minkä seurauksena sanasta ”walking” saadaan sana ”walk” (Martin, 2023). Esimerkki stemmattavasta lauseesta voisi olla seuraavanlainen: ”I am walking to my working place”, joka stemmataan muotoon ”I am walk to my work place”. Manning (2009) kuvaa stemmausta karkeana heuristisena prosessina, joka onnistuu tavoitteessaan riittävän usein. Kokash (2005) kertoo, että heuristisen prosessin tavoite ei ole pyrkiä täydellisyyteen, vaan likimääräiseen ratkaisuun. Hän mainitsee, että likimääräiseen ratkaisuun pääseminen riittää, sillä monimutkaisten ongelmien täydellinen ratkaiseminen voi olla saatavilla olevaan aikaan verrattuna mahdotonta. Stemmaaminen voi päätyä usein tilanteeseen, jossa takaliitteen poistaminen muuttaa sanan tarkoituksen (Martin, 2023). Jos sääntönä on poistaa ”-er” päätteet sanasta ”better” tulee sana ”bett”. Kyseisessä tilanteessa sanan merkitys ei enää säily. Vijayarani ja muut (2015) kuvaavat useiden stemmaajien olevan kielestä riippuvaisia. Stemmaajat, jotka ovat tehty englannin kielelle eivät siis toimi suoraan suomen kielelle. Porterin stemmaajaa kuvataan yksinkertaisena ja tehokkaana stemmaajana, muttei kovin tarkkana (Martin, 2023). Manning (2009) huomioi stemmaajan olevan puutteellinen, koska mainitsee toisen stemmaajan tekevän asiat oikein.

Vijayarani ja muut (2015) mainitsevat vaihtoehtoisiksi stemmaajiksi muun muassa Lovinsin stemmaajan ja N-gram stemmaajan. He noteeraavat, että Lovinsin stemmaajaa pidettiin ensimmäisenä tehokkaana stemmajana 1960-luvun lopulla. He kertovat, että toimintaperiaate perustuu pisimmän takaliitteen poistamiseen sanasta. Näin voidaan kategorisoida Lovinsin stemmaaja Porterin stemmaajan tavoin katkaisumenetelmiin. Vijayarani ja muut (2015) mukaan tilanteessa, jossa pisin takaliite on poistettu, niin Lovinsin stemmaaja hienosäätää typistetyt sanat, jotta sanoista muodostuu oikeita. Esimerkiksi sanasta ”happiness” Lovinsin stemmaajaa poistaa suurimman takaliitteen ”-ness” ja lopputuloksena on ”happi”. Tällöin hienosäätövaiheessa ”happi” muutetaan oikeaan sanaan ”happy” käyttämällä sääntöä, jossa i-kirjain muutetaan y-kirjaimeksi. Vijayarani ja muut (2015) kertovat, että hienosäätövaihe puuttuu aiemmin mainitusta Porterin stemmaajasta, jossa takaliitteen poistaminen voi johtaa sanaan, joka ei tarkoita mitään. He lisäävät, että Lovinsin stemmaaja on kuitenkin aikaa vievä ja hienosäädöstä huolimatta sitä pidetään epäluotettavana kantasanojen saamisen suhteen.

Vijayarani ja muut (2015) mainitsevat, että tilastollisiin stemmaajiin kuuluu N-gram stemmaaja, jota pidetään kielestä riippumattomana stemmaajana. He kertovat, että N-gram stemmaaja perustuu vierekkäisiin merkkeihin ja käyttää kantasanan löytämiseen periaatetta, jossa samankaltaisilla sanoilla on korkea N-gram pitoisuus. N-gram sanat muodostetaan päättämällä ensin n-arvo. Vijayarani ja muut (2015) mukaan n-arvo on tyypillisesti neljä tai viisi. Esimerkkiä varten valitaan n-arvoksi 3. Tällöin sanasta ”processing” saadaan N-gramit: [’pro’, ’roc’, ’oce’, ’ces’, ’ess’, ’ssi’, ’sin’, ’ing’]. Kantasana saadaan erottamalla saman sanan esiintymien jälkiliitteet (Engati, 2023). Tällöin voidaan päästä tilanteeseen, jossa ”processing” päätyy sanaan ”process”. Vijayarani ja muut (2015) kertovat, että kyseisen stemmaajan etuna pidetään sen kielestä riippumattomuutta, mutta negatiivisena vaikutuksena N-gramien säilyttäminen muistissa sekä niiden indeksointi vie paljon tilaa. He mainitsevat myös, että lähestymistapaa kuvataan sen hyödyllisyydestä huolimatta ei-käytännöllisenä muun muassa sen muistivaatimuksen takia.

Vijayarani ja muut (2015) informoivat, että hybridimenetelmiin on olemassa korpukseen perustuvia stemmaajia. He kertovat, että korpukseen perustuvat stemmaajat pyrkivät pantamaan Porterin stemmaajan huonoja puolia, jossa stemmatut sanat voivat muodostaa samasta kantasanasta eri tarkoituksen omaavia sanoja. Korpusta hyödyntävät stemmajat keräävät kontekstiin liittyvän korpuksen, jota käytetään analysoimaan samaan kantasanaan stemmattuja sanoja (Vijayarani ja muut, 2015). Vijayarani ja muut (2015) noteeraavat, että kantasanojen analysointi mahdollistaa oikean kantasanan löytämisen kontekstin avulla. He kertovat, että analysointi suoritetaan tilastollisia menetelmiä käyttämällä. Koska tilastolliset menetelmät ovat riippuvaisia korpuksesta, heidän mukaansa stemmaajan huono puoli on tarve prosessoida jokainen korpus erikseen.

2.1.2 Lemmatisointi

Manning (2009) kuvaa lemmatisointia prosessina, joka selvittää sanan perusmuodon oikein. Khyani, Siddhartha, Niveditha ja Divya (2021) kertovat prosessin perustuvan ideaan, jonka tarkoitus on selvittää sanan alkuperäinen merkitys. Eroavaisuutena stemmaukseen, lemmatisoija pitää yllä viitteitä sanojen taivutetuista muodoista ja niiden perusmuodoista (Martin, 2023). Sanan perusmuotoja kutsutaan kielitieteessä lemmoiksi (Khyani ja muut, 2021). Kuten preesensmuoto: ”walking” → lemma:”walk”. Viitteiden ylläpitoon voidaan ajatella avainarvo tietorakenne, jossa avaimina ovat taivutetut sanat ja arvoina ovat taivutetun sanan perusmuodot. Tällöin luvun 2.1.1 esimerkistä saadaan: ”I be walk to my work place”. Manning (2009) mainitsee, että lemmatisoinnin huonona puolena pidetään tilavaativuutta sekä morfologista analysointia. Morfologinen analyysi auttaa hahmottamaan muun muassa morfeemien semanttista merkitystä. Manning (2009) kertoo kuitenkin, että morfologisen analysoinnin yhdistäminen hyvään sanastoon antaa suuret mahdollisuudet löytää sanan lemma. Hän lisää myös, että morfologinen analysointi tuo marginaalisia parannuksia, mutta tuo suorituskyvyllisiä hankaluuksia sitä hyödyntäville sovelluksille.

Plisson, Lavrac ja Mladenic (2004) esittävät, että eräs lemmatisoinnin lähestymistapa on loppupäätteiden korvaukset. He kertovat tämän prosessin keskittyvän sääntöjen löytämiseen, joiden avulla loppupäätteet voidaan korvata oikein. He myös korostavat, että sääntöjen löytämisen haastetta voidaan tarkastella koneoppimisen näkökulmasta. Koulutusdatassa sanat esitetään niiden taivutusmuodoissa (Plisson ja muut, 2004). Koulutusdata on puolestaan joukko ominaisuuksia, jotka ovat mahdollisesti riippuvaisia merkinnästä. Koneoppimisessa koulutus tarkoittaa sopivien riippuvuuksien löytämistä, joiden avulla voidaan saada oikea merkintä uudelle datalle. Plisson ja muut (2004) tarkentavat, että jokaiselle sanalle annetaan merkintä, joka ilmaisee, kuinka sen loppupäätteet tulisi korvata. Esimerkiksi, kun otetaan huomioon sana ”mice”, sen kantasana on ”mouse”. Tästä seuraa, että merkintä on ”iceTOouse”, koska sanojen ainoa yhteinen osa on ”m”, ja loput kirjaimet korvataan.

Toinen lähestymistapa on Ripple Down Rule -lähestymistapa. Plisson ja muut (2004) kertovat lähestymistavan olevan sääntöihin pohjautuva, ja että säännöt ilmaistaan ”if-else” rakenteella.

if $i = VERB$ **then**
    if $VERB = TYPE(PAST)$ **then**
        $i \leftarrow remove \leftarrow SUFFIX(-ed)$
    else if $VERB = ENDS(-ied)$ **then**
        $i \leftarrow replace \leftarrow SUFFIX(-ied) \leftarrow SUFFIX(-y)$

end if
else
continue
end if

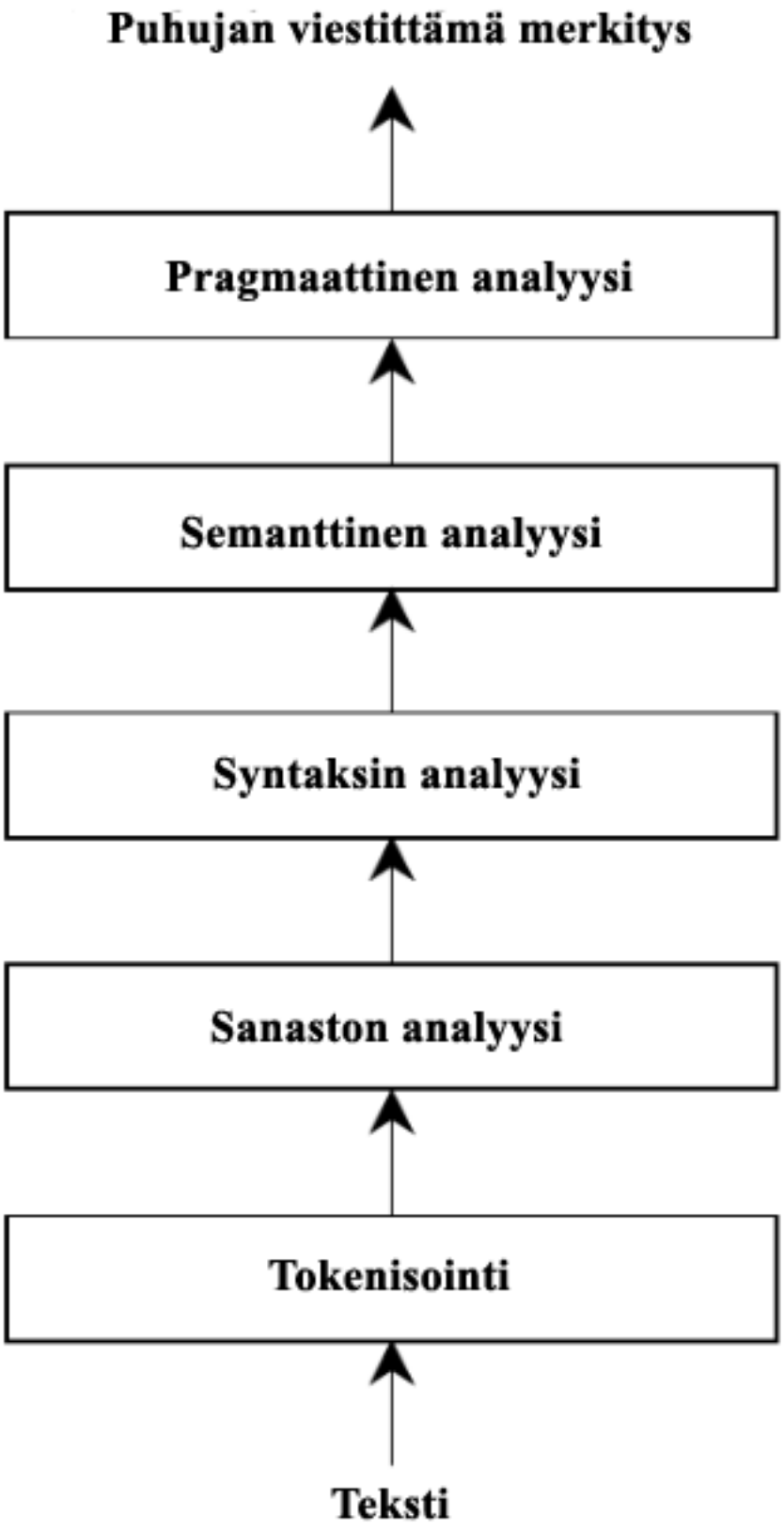
Algoritmi toteuttaa säännön, jossa tarkastellaan, onko sana verbi. Jos sana on verbi, niin poistetaan sanasta loppupääte ”-ed”. Toisaalta, jos sanan loppupääte on ”-ied”, niin sanan pääte korvataan päättellä ”y”. Muuten suorittamista jatketaan. Algoritmista nähdään selkeä sääntöihin pohjautuva ajattelutapa, jolla voidaan korvata tai poistaa sanojen loppupäätteitä, ja muuttaa sana kantasanaksi.

Plisson ja muut (2004) esittävät, että lähestymistavan säännöt luodaan siten, että niihin sisällytetään poikkeuksia. He korostavat, että nämä poikkeukset rajoittuvat omaan kontekstiinsa, eivätkä ne vaikuta muihin sääntöihin. He mainitsevat ohjenuoraksi seuraavan: kun uusia sääntöjä lisätään ja mikäli sääntö aktivoituu mutta tuottaa väärän tuloksen, käytetään tarkennusta **else if**. Jos sääntöhaaraan ei päästä lainkaan, sääntö sijoitetaan **else**-haaraan. Plisson ja muut (2004) kertovat, että sääntöjen luominen alkaa yleisimmistä tapauksista ja laajenee, kun poikkeustapauksia ilmenee. He mainitsevat, että tämä lähestymistapa sopii hyvin lemmatisointiin, mutta yleisenä huomiona he pitävät ylläpidettävyyden haasteita.

2.2 Luonnollisen kielen analyysikeinot

Indurkhya ja Damerau (2010) kertovat, että luonnollisen kielen tekstin analysointi voidaan karkeasti jakaa syntaktiseen, semanttiseen ja pragmaattiseen analysointiin. He selventävät, että tämä jako on yksinkertaistus analysoinnin vaiheista, mutta se antaa vakaan pohjan jatkotyölle. Syntaksi keskittyy sanojen järjestykseen ja rakenteeseen, semantiikka sanojen tarkoitukseen ja pragmatiikka tekstin kokonaismerkitykseen (Indurkhya & Damerau, 2010). Vaikka luonnollisen kielen analysointi on monimutkainen prosessi, Indurkhya ja Damerau (2010) jakavat sen eri vaiheisiin.

Kuvassa 2.1 sanaston analyysi on seuraava vaihe tokenisoinnin jälkeen. Sanaston analyysi keskittyy yksittäisten sanojen ja niiden merkitysten tutkimiseen (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) kertovat, että sanaston analyysi käsittää sanojen tunnistamisen ja luokittelun, morfologisten rakenteiden analysoinnin ja leksikaalisten ominaisuuksien ymmärtämisen. Leksikaalisia ominaisuuksia ovat muun muassa sanaluokat, sanan morfologia ja semantiikka (Yule, 2022). Indurkhya ja Damerau (2010) mainitsevat, että yhtenä tavanomaisena tehtävänä sanaston analyysissä pidetään



Kuva 2.1: Luonnollisen kielen analyysin vaiheet. Muokattu lähteestä (Indurkhya & Damerau, 2010).

Taulukko 2.1: Taulukko sanan semanttisista ominaisuuksista. Muokattu lähteestä (Yule, 2022).

	horse	boy	man	girl	woman
animate	+	+	+	+	+
human	-	+	+	+	+
female	-	-	-	+	+
adult	+	-	+	-	+

morfologisten varianttien yhdistämistä niiden lemmoihin. He kertovat, että lemmat ovat usein tallennettuna sanakirjaan, jossa on myös sanan muuttumattomat semanttiset ja syntaattiset tiedot. He myös mainitsevat, että eräänä sanaston analyysin sovelluksena olevan tekstin esikäsittely, jota hyödynnetään syntaksin analyysissä.

Syntaksin analyysi pohjautuu sanojen sijoittelun ja rakenteen merkitykseen lauseissa (Yule, 2022). Syntaksin analyysistä käytetään usein myös termiä syntaksin parsiminen (Indurkhya & Damerau, 2010). Yule (2022) kertoo, että lauseiden merkitystä selvitetään analysoimalla sanojen välisiä suhteita ja riippuvuuksia. Hän myös lisää, että syntaksin analyysi keskittyy lauseen määrittelyyn ja hierarkkiseen ymmärtämiseen.

Semanttinen analyysi puolestaan keskittyy sanojen ja lauseiden analysointiin annetussa kontekstissa (Indurkhya & Damerau, 2010). Semantiikka nimensä mukaan pyrkii selvittämään merkitystä. Yule (2022) tekee huomion, että useille sanoille ei ole helppoa määrittää selkeitä merkityksen antavia komponentteja. Indurkhya ja Damerau (2010) kertovat semanttisen analyysin keskittyvän ymmärtämään sanojen ja niiden tulkintojen välisiä suhteita. He lisäävät, että semanttiseen analyysiin kuuluu sanojen yhdistäminen niiden ontologioihin sekä vastaaviin käsitteisiin. Sanan semanttisia ominaisuuksia voidaan erotella kuten nähdään taulukossa 2.1.

Pragmaattinen analyysi keskittyy kielen käyttöön tietyssä kontekstissa (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) kertovat, että kontekstin muutos voi muuttaa kielen todellista tulkintaa. He korostavat myös, että kielen tulkintaan vaikuttavat puhujan tiedot, aikomukset ja uskomukset. He lisäävät, että pragmaattinen analyysi mahdollistaa tekstin oikean tulkinnan. Yule (2022) mainitsee, että analyysi keskittyy usein näkymättömän merkityksen tutkimiseen, eli sanojan alkuperäisen tarkoituksen selvittämiseen. Indurkhya ja Damerau (2010) toteavat, että puhujien on voitava luottaa moniin jaettuihin oletuksiin ja odotuksiin kommunikoidessaan. Esimerkiksi lause ”We are in the same boat”, ei aina tarkoita kirjaimellisesti fyysistä sijaintia. Lauseen kontekstin analysointi puolestaan auttaa tulkitsemaan tarkoitetun merkityksen.

2.3 Lähestymistavat luonnollisen kielen oppimiseen

Wermter, Riloff ja Scheler (1996) mainitsevat luonnollisen kielen oppimisen saavuttaneen kehitystä ja uusia näkökulmia. He kuvaavat luonnollisen kielen oppimista tärkeänä automaattisen tiedonhankinnan työkaluna, jota nähdään muun muassa puheen tunnistuksessa, konekääntämisessä ja tiedonhaussa. Esimerkkinä tiedonhankinnassa on tärkeää päästä moniselitteisyydestä eroon. Jackson ja Moulinier (2002) kertovat, että epäselvissä tilanteissa, jossa esiintyy mahdollisesti paikan- tai henkilön nimi, on tapana valita henkilön nimi. He huomioivat myös, jos maantieteellinen sijainti on lähellä mahdollista paikannimeä, niin luokitellaan se tällöin paikannimeksi. Wermter ja muut (1996) mainitsevat, että luonnollisen kielen oppimisen hyödyntäminen mahdollistaa skaalautuvien, joustavien ja kannettavien luonnollisen kielen järjestelmien rakentamisen. Liddy (2001) kuvaa luonnollisen kielen oppimisen jakautuvan kolmeen erilaiseen lähestymistapaan.

Liddy (2001) kuvaa symbolisen lähestymistavan (engl. *symbolic approach*) perustuvan ihmisten luomiin sääntöihin ja sanastoihin. Hän mainitsee lähestymistavan analysoivan kieltä tietoskeemojen ja algoritmien avulla. Wermter ja muut (1996) kertovat symbolisen lähestymistavan koostuvan induktiivisista koneoppimisen tekniikoista, kuten päätöspuut ja käsitteellinen ryhmittely. He kertovat myös symbolisiin lähestymistapoihin kuuluvan selitys- sekä tapauspohjainen oppiminen. Selityspohjainen oppiminen luo yleistettyjä esimerkkejä tulevaa varten, ja tapauspohjainen oppiminen yrittää karttaa edellä koettuihin tilanteisiin uusia mahdollisuuksia (Wermter ja muut, 1996). Lähestymistapaa nähdään usein logiikka- ja sääntöpohjaisissa järjestelmissä (Liddy, 2001).

Liddy (2001) mainitsee tilastollisen lähestymistavan (engl. *statistical approach*) hyödyntävän matemaattisia tekniikoita, joiden avulla voidaan luoda yleistettyjä malleja. Hän mainitsee, että yleisten mallien avulla voidaan kuvata kielen piirteitä, jotka pohjautuvat annettuun korpukseen. Malleja voidaan kuvata työkaluina, jotka auttavat tekemään saatavilla olevan informaation perusteella valistuneita arvauksia (Jackson & Moulinier, 2002). Liddy (2001) kuvaa tunnetuimmaksi tilastolliseksi malliksi Markovin piilomallin (engl. *hidden Markov model*). Hän tarkentaa, että mallia kuvaillaan äärellisenä tilakoneena, ja että tilojen väliset siirtymät kuvaavat todennäköisyyttä siirtyä kyseisiin tiloihin. Wermter ja muut (1996) tekivät pohjatyön, joka tukee Liddyn (2001) esitystä kuinka tilastollisia lähestymistapoja kuvataan suurilla korpuksilla. He lisäävät, että tilastollisen mallin tunnusmerkkinä voi olla analyysin suorittaminen keskittyen tekstin ominaisuuksiin, ilman suurempaa lisäystä kielelliseen tietämykseen. Korpusten merkitseminen sanaluokilla tai jäsennyspuilla on ollut tärkeä osa korpuspohjaista kielianalyysia (Wermter ja muut, 1996). Wermter ja muut (1996) kertovat tilastollisen analyysin mahdollistaneen

sanastojen luonnin leksikaalisella ryhmittelyllä sekä todennäköisyyksiin perustuvan kieliopin oppimisella. He mainitsevat metodeiksi muun muassa N-gramit, ohjaamattoman ryhmittelyn sekä Markovin piilomallit.

Liddy (2001) kuvaa konnektionistisen lähestymistavan (engl. *connectionist approach*), tilastollisen lähestymistavan tavoin, luovan yleistettyjä malleja kielellisistä ilmiöistä. Hän mainitsee konnektionistisen lähestymistavan eroavan tilastollisesta lähestymistavasta muun muassa lisäämällä esitysteorioita, jotka mahdollistavat syvemmän ymmärryksen tiedon kulkuun. Lähestymistavalle on ominaista toisiinsa kytkettyjen prosessointiyksiköiden verkko, jossa tietoa tallentuu yksiköiden välisiin yhteyksiin (Liddy, 2001). Wermter ja muut (1996) kertovat konnektionististen verkkojen mallintaneen onnistuneesti kielen oppimisen liittyviä tehtäviä. He huomioivat, että konnektionismin yhdistäminen tilastollisiin sekä symbolisiin esityksiin on esittäytynyt tärkeäksi tutkimuksen kentäksi. Liddy (2001) jakaa konnektionistisen lähestymistavan lokalistisiin ja jakautuneisiin malleihin. Hän kuvaa lokalististisen mallien yksittäisen yksikön kuvastavan yhtä konseptia, ja konsepti voi tarkoittaa esimerkiksi eläimen käsitettä. Jakautuneiden mallien kohdalla hän kertoo useiden yksikköjen aktivoitumisen muodostavan yhden konseptin.

Näiden eri lähestymistapojen käyttämisessä noudatetaan yleistä vaiheiden joukkoa: datan keräämistä, datan analyysia, sääntöjen rakentamista ja niiden soveltamista järjestelmässä (Liddy, 2001). Hän kuvaa, että vaiheet voidaan suorittaa ilman koneoppimisen keinoja tai niiden kanssa. Liddy (2001) kertoo datan keräämisvaiheen olevan tärkeä kaikille kolmelle lähestymistavalle, mutta tilastolliset ja konnektionistiset lähestymistavat vaativat yleensä enemmän dataa kuin symboliset lähestymistavat. Datan analysoinnin vaiheessa hän mainitsee, että symboliset lähestymistavat nojaavat ihmisten tekemään analyysiin teorioiden muodostamiseksi. Hän jatkaa, että symbolinen malli määritellään toisaalta manuaalisesti ja konnektionistinen malli rakennetaan datasta. Sääntöjen rakentamisvaiheessa manuaalinen työ on tyypillistä symbolisille lähestymistavoille. Tilastolliset ja konnektionistiset lähestymistavat käyttävät tilastollista tai konnektionistista mallia ja rakentavat sääntöjä tai datakohteita automaattisesti (Liddy, 2001). Liddy (2001) huomioi, että tilastolliset metodit perustuvat symbolisiin malleihin ja tilastollinen lähestymistapa itsessään ei ole riittävä. Hän huomioi myös, että lähestymistapoja on hyödyllistä pitää toisiaan täydentävinä, siksi hybridiratkaisut ovat saaneet kiinnostusta luonnollisen kielen ongelmien parissa.

2.4 Säännölliset lausekkeet

Martin (2023) kertoo, että säännöllisiä lausekkeita käytetään muun muassa kuvaamaan

haettavien merkkijonojen rakenteita. Hän mainitsee myös, että säännölliset lausekkeet ovat hyödyllisiä tekstistä hakemiseen ja niitä käytetään usein korpuksien läpikäymisessä. Säännöllisiä lausekkeita voidaan käyttää esimerkiksi tekstin hakemisen, palauttamaan kaikki syntaksiin sopivat merkkijonot, sekä lausekkeita voidaan käyttää tekstin esikäsittelyyn ja poistamaan tarpeettomia merkkijonoja. Säännölliset lausekkeet ovat laskennallisesti tehokkaita ja helposti muunneltavia haluttuun käyttötarkoitukseen (Martin, 2023). Martin (2023) kuvaa, että lausekkeiden säännöt voivat perustua merkin tai merkkijonon vierekkäisyyteen, toistoon tai vuorotteluun. Säännöllisten lausekkeiden käyttäminen ei ole sidonnaista ohjelmointikieleen ja niiden käyttö on universaalia (Jackson & Moulinier, 2002). Martin (2023) noteeraa säännöllisten lausekkeiden olevan huomaamaton standardisaation menestys. Murthy, Padmanabhan ja Deshpande (2012) huomioivat rakenteettoman datan kasvavan tärkeyden ja noteeraavat säännöllisten lausekkeiden olevan yksi tärkeä työkalu tiedonhakujärjestelmille. Tiedonhakujärjestelmiä esitellään tarkemmin luvussa 2.5.

Yksinkertaisimmillaan lauseke voi olla esimerkiksi */book /* , jolloin ensimmäinen ”book”-sanan sisältävä merkkijono palautetaan. Lausekkeet ovat kirjainkokoherkkiä (Martin, 2023). Tämä voidaan ottaa huomioon käyttämällä hakasulkuja */[Kk] irja/*, jolloin voidaan palauttaa kirjainkokoherkät esiintymät. Hakasulut kuvaavat merkkien disjunktion (Martin, 2023). Hakasulkujen sisään voidaan asettaa suljettu väli (–) , jolloin palautettava merkkijono on jokin suljetulla välillä esiintyvä merkkijono (Martin, 2023). Esimerkiksi */[a–z]/* palauttaa ensimmäisen a:n ja z:n välillä esiintyvän merkin. Martin (2023) mainitsee, että säännölliset lauseet tukevat myös negaatioita käyttämällä merkkiä ^, jolloin seuraava lauseke */[^a–z] /*, ei palauta ensimmäistä pientä merkkiä. Lisäksi hän huomioi, että merkin sijainnilla on merkitystä, jos merkin asettaa */[a ^] /* esimerkiksi a:n jälkeen, niin seuraava lauseke palauttaa a:n tai ^.

Säännölliset lausekkeet voivat palauttaa vääriä positiivia sekä vääriä negatiivisia tuloksia. Väärät positiiviset tulokset ovat sellaisia, joissa haettu merkkijono ei ole toivotun mukainen. Kyseinen tilanne voi esimerkiksi päätyä hakemalla sanaa ”cat”, mutta säännöllinen lauseke palauttaa myös sanan ”catalog”. Tilanteisiin voi varautua määrittämällä säännöllisen lausekkeen tarkemmin, esimerkiksi lauseke */\btea\b/*, jossa \b tarkoittaa sanan rajaa. Väärät negatiiviset ovat toisaalta sanoja, jotka ovat jääneet tuloksista pois virheellisesti. Tällaisia tilanteita voi tapahtua esimerkiksi, jos sanan esiintymistä lauseen ensimmäisenä, kuten ”Tea”, ei huomioida. (Martin, 2023) Murthy ja muut (2012) noteeraavat säännöllisten lausekkeiden yleistämisen olevan haastava tehtävä. He mainitsevat myös, että kattavien säännöllisten lausekkeiden johdoksi kohdataan usein vääriä positiivisia ja vääriä negatiivisia tuloksia.

2.5 Käyttökohteet

Martin (2023) listaa luonnollisen kielen käsittelyn esiintyvän usein seuraavissa sovelluksissa:

- Tiedonhaku (engl. *information retrieval*)
- Tiedonpoiminta (engl. *information extraction*)
- Kysymyksiin vastaaminen (engl. *question-answering*)
- Tekstin tiivistäminen (engl. *summarization*)
- Konekääntäminen (engl. *machine translation*)
- Dialogijärjestelmät (engl. *dialogue systems*)

Tiedonhaku pyrkii etsimään asiakirjoja perustuen käyttäjän tarpeisiin (Martin, 2023). Asiakirjat voivat olla mitä tahansa tekstimuotoista dataa, kuten tieteelliset artikkelit, keskustelupalstat ja verkkosivut. Tiedonhakujärjestelmät ovat useimmiten tunnettu termillä hakukone (Martin, 2023). Tiedonhakujärjestelmiä kuvataan tarkemmin luvussa 3.1.

Tiedonpoiminnan prosessi muuttaa rakenteettoman tiedon rakenteelliseksi tiedoksi (Martin, 2023). Esimerkiksi tapahtuman uutisartikkeli voidaan erotella tietokantaan louhimalla tekstistä tapahtumapaikka, päivämäärä ja tapahtuman nimi. Kysymyksiin vastaaminen vie tiedonhaun askeleen pidemmälle ja tarjoaa käyttäjälle suoraan vastauksen, mahdollisten vastausjoukkojen sijaan (Martin, 2023). Kysymyksiin vastaamista käsitellään ympäri tutkielmaa, ja esitetyt aihepiirit liittyvät mahdollisiin kysymyksiin-vastaamisjärjestelmiin.

Mishra ja muut (2014) kuvaavat tekstin tiivistämistä menetelmänä, jonka päämääränä on esittää tekstin ydinaihe tuottamalla tekstimuotoinen tai graafinen tiivistelmä. Tiivistelmät auttavat käyttäjiä saavuttamaan tärkeän tiedon nopeasti, sekä potentiaalisesti tarkemmin kuin itse etsimällä. Tekstin tiivistäminen on erityisen hyödyllistä muun muassa tutkijoille, jotka käsittelevät paljon uutta tietoa (Mishra ja muut, 2014). Mishra ja muut (2014) huomioivat yhteenvetoamisen olevan riippuvainen käytettävistä korpuksista ja niiden kielestä. He kertovat, että englannin kieltä pidetään luotettavimpana kielenä tekstin käsittelylle, sillä kielelle on tehty eniten kielen analysointiin vaadittuja työkaluja.

Martin (2023) kertoo konekääntämisen olevan yleinen luonnollisen kielen käsittelyn tehtävä, jonka avulla edistetään informaation saantia. Hän myös mainitsee, että suuri

määrä informaatiota on kerätty eri kielille, mutta erityisesti informaatiota on saatavilla englannin kielelle. Informaation määrän eroavaisuutta hän esittää termillä digitaalinen jakauma. Tämän vuoksi konekääntämisestä on tullut tärkeää ja Martin (2023) arvioi Google kääntäjän kääntävän satoja miljardeja sanoja päivässä. Korkealaatuiset käännökset ovat tärkeitä kielille, joita kutsutaan matalaresurssisiksi (Martin, 2023). Hän kertoo hetkessä kääntämisen (engl. *on-the-fly translation*) olevan tuoreimpia konekääntämisen sovelluksia. Kääntäminen tapahtuu kuvauksen mukaan samanaikaisesti, kun puhetta tai tekstiä on saatavilla.

Martin (2023) kuvaa dialogia yhtenä kielen perustana sekä kertoo sen olevan ihmisille yleisin kommunikaation keino. Hän tiedottaa dialogijärjestelmien olevan luonnollisen kaltainen tapa päästä haluttuun päämäärään, jonka avulla voidaan hakea tietoa tai suorittaa koneellisia toimintoja. Nykyisiä kaupallisessa omistuksessa olevia dialogijärjestelmiä ovat muun muassa Applen Siri ja Amazonin Alexa (Martin, 2023).

3. Kysymyksiin vastaamisen menetelmät

Kysymyksiin vastaamisen menetelmät ovat tärkeä osa prosessia, jonka avulla voidaan luoda annetusta kysymyksestä vastaus, kuten Jackson ja Moulinier (2002) toteavat. Jackson ja Moulinier (2002) pitävät kysymyksen ymmärtämistä olennaisena osana luoda oikea vastaus. Heidän mukaansa luonnollisen kielen ymmärtäminen pyrkii pääsemään tavoitteeseen, jossa tietokoneet pystyvät ymmärtämään luonnollista kieltä ihmisen tavoin. He pitävät tavoitetta hankalana muun muassa määrittelyongelman vuoksi, kuinka määritellään ymmärtämisen raja. Jackson ja Moulinier (2002) vertaavat koneellista luonnollisen kielen ymmärrystä tietokoneen tietoisuudeksi. He näkevät tämän olevan utopistinen näkökulma, tai dystopinen, riippuen suhtautumisesta. Lisäksi he ehdottavat olevan totuudenmukaisempaa puhua tiedon poimimisesta. DARPA (Defense Advanced Research Projects Agency) järjesti sarjan työpajoja 1990-luvulla, kerätäkseen tietoa rakenteettomasta datasta, kuten Jackson ja Moulinier (2002) kertovat. Jackson ja Moulinier (2002) mainitsevat yhden työpajan tavoitteena olleen kerätä englanninkielisistä terrorismia käsittelevistä uutisartikkeleista ominaisuuksia kuten aika, paikka ja kohteena oleva uhka. He raportoivat kyseisen työpajan saaneen 50 % ominaisuuksista kerättyä ja 60 % yleisestä tapahtuman tarkkuudesta. He toteavat eri työpajojen päässeen korkeimmillaan 75 % tarkkuuksiin. Erilaisia keinoja, joiden avulla poimittiin tietoa, olivat hahmojen sovittaminen tekstiin, äärelliset tilakoneet, kontekstivapaa jäsentäminen ja tilastollinen mallinnus (Jackson & Moulinier, 2002).

Seuraavissa aliluvuissa esitellään tiedonhakujärjestelmät, jotka hyödyntävät erilaisia esitettyjä tiedonhakumalleja, jotka mahdollistavat kysymyksiin vastaamisen tiedonhaun. Sanaluokkien tunnistaminen auttaa ymmärtämään lauseen rakenteellista merkitystä. Entiteettien yhdistäminen mahdollistaa tekstissä esiintyvien sanojen yhdistämisen oikeisiin entiteetteihin, joka auttaa välttämään väärinkäsityksiä. Tietopohjaiset kysymyksiinvastaamisjärjestelmät ovat klassinen vaihtoehto kysymyksiin vastaamiselle, jotka hyödyntävät muun muassa entiteettien yhdistämistä.

3.1 Tiedonhakujärjestelmät

Indurkhya ja Damerau (2010) noteeraavat, että tiedonhakujärjestelmiä pidetään onnistuneina luonnollisen kielen käsittelyn sovelluksina. He mainitsevat yleisimpinä tiedonhakujärjestelmien sovelluksina olevan hakukoneet. He kuvailevat, että tiedonhakujärjestelmiä voidaan kuvata seuraavien ominaisuuksien kautta: järjestelmät vastaavat tiedon esittämisestä, varastoinnista, järjestämisestä ja saavuttamisesta. Lisäksi he nostavat tärkeän ongelman ratkaistavaksi, joka on epätäydellisen kyselyn käsittely. Indurkhya ja Damerau (2010) huomauttavat, että käyttäjät yleensä jättävät kyselyistä pois tarkentavat lisätiedot ja haluavat järjestelmän ehdottavan vastauksia. He kuvaavat hakuprosessia yrityksen ja erheen metodina, kunnes päästään sopivaan vastaukseen. Haku on myös haastava polysemian eli sanojen monimerkityksellisyyden takia, kuten Indurkhya ja Damerau (2010) toteavat. He myös huomioivat, että sanojen synonyymit eivät helpota oikean tiedon saatavuutta. Indurkhya ja Damerau (2010) kertovat, että hakujärjestelmät toimivat kyselyn vertaamisella asiakirjoihin, ja asiakirjoja, jotka sisältävät eniten samoja termejä kyselyn kanssa, pidetään parhaina tuloksina. He mainitsevat, että tiedon hakeminen suuresta dokumenttipankista on raskasta, joten haku sellaisenaan ei ole suotavaa.

Ratkaisuna haun tehokkuusongelmiin on indeksointi, kuten Indurkhya ja Damerau (2010) mainitsevat. He tarkentavat, että tiedonhakujärjestelmät eivät suoraan hae tietoa dokumenttipankista vaan hakevat tietonsa indeksistä. Indurkhya ja Damerau (2010) toteavat, että indeksi pyrkii säilyttämään tiedonhakuun olennaisia ominaisuuksia, jotta kokonaista dokumentin läpikäyntiä ei tarvita. He kuvaavat, että indeksissä voidaan säilyttää yksittäisiä sanoja tai yhdistelmiä sanoista ja tilanteen mukaan sanoja voidaan myös pilkkoa pienempiin osiin. Esimerkiksi eräs kieliriippumaton indeksointitapa on N-gram indeksointi, jossa N-gram indeksoinnissa päätetään n muuttujan arvo, esimerkiksi $n = 4$. Tällöin lauseen ”Beautiful weather outside” indeksit ovat: Beau, eaut, auti, utif, ... , side. Indurkhya ja Damerau (2010) suosittelevat N-gram indeksoijaa kielille, joille ei ole morfologisia analysaattoreita, ja huomauttavat, että kyseinen indeksoija on myös hyödyllinen, kun teksti sisältää paljon kirjoitusvirheitä. He korostavat indeksoinnissa olevan myös tärkeää rajata olennaiset ominaisuudet, ja ominaisuuksien määrittämistä varten voidaan ottaa huomioon myös asiakirjojen tärkeys. Indurkhya ja Damerau (2010) mainitsevat, että tärkeämpi dokumentti voi olla tarkemmin indeksoitu, ja vaikka indeksoinnissa yritetään keskittyä yhdenmukaisuuteen, eri indeksoijat käyttävät eri avainsanoja. He kertovat tämän korostavan lukuisia eri tapoja kuvata samaa asiaa, mutta huolimatta vaihtelun välttämättömyydestä, on suotavaa pyrkiä yhdenmukaisuuteen.

Indurkhya ja Damerau (2010) huomioivat, että vaikka manuaalisella indeksoinnilla

on mahdollista kasvattaa asiakirjapankin yhdenmukaisuutta, suurin osa tiedonhakujärjestelmistä käyttää automaattista indeksointia. He kertovat, että yksinkertainen indeksointialgoritmi noudattaa neljää eri vaihetta: rakenteen analyysi, pysäytyssanojen poistaminen, morfologinen normalisointi ja painotus. He tarkentavat, että rakenteen analysoinnin tarkoitus on tunnistaa asiakirjan rakenne ja vain olennaiset rakenteenosat jatkavat käsittelyä. Olennainen rakenteenosa voi olla esimerkiksi tieteellisen artikkelin tiivistelmä. Käsittelyyn jatkaneet rakenteet tokenisoidaan (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) jatkavat, että tokenisoinnissa olevista sanoista poistetaan pysäytyssanat. He tarkentavat, että pysäytyssanat ovat tekstissä usein esiintyviä sanoja kuten englanninkieliset sanat ”the” ja ”a”. Suomenkielisiä pysäytyssanoja ovat muun muassa ”ja” ja ”on”. Saini ja Rakholia (2016) mainitsevat, että jos pysäytyssanoja kohdellaan funktiosanoina, silloin saatetaan pitää normaalisti poistettavat pysäytyssanat. He kertovat, että funktiosanoina pidetään sellaisia sanoja, jotka muodostavat kieliopillisia tai rakenteellisia suhteita sanojen välillä. Esimerkiksi englannin kielen sana ”to” luo suhteen sanojen välille. Pysäytyssanat poistetaan yleensä siksi, koska ne eivät sisällä hakemiseen liittyvää merkityksellistä tietoa (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) huomioivat, että pysäytyssanat voivat pahimmillaan haitata hakutuloksia lisäämällä samankaltaisuutta ei-olennaisten asiakirjojen kanssa. He lisäävät, että pysäytyssanojen poistaminen myös vähentää indeksissä olevan tiedon määrää huomattavasti pysäytyssanojen yleisyyden vuoksi, mutta poistettavien pysäytyssanojen valitseminen voi olla haastavaa. Heidän mukaansa valikointiin ei ole selvää ohjetta. Saini ja Rakholia (2016) tukevat edellä mainittua väitettä ja mainitsevat mahdollisuudeksi yhdistää useita pysäytyssanalistoja korvaamaan standardien puutetta. Lisäksi he mainitsevat pysäytyssanalistan olevan suomessa korkeintaan 747 sanaa pitkä. He kertoivat myös, että lyhyin lista tutkituista kielistä annettiin Ukrainalle 28 pysäytyssanalla. Suomen kielellä on tavallista määrää enemmän takaliitteitä kuin muilla Indo-Eurooppalaisilla kielillä (Korenius, Laurikkala, Järvelin & Juhola, 2004). Tämä nähdään pysäytyssanalistojen pituuksissa. Toisena huomiona Saini ja Rakholia (2016) pitävät pysäytyssanalistan pituutta merkkinä luonnollisen kielen käsittelyn algoritmien kelpoisuudesta. Jos poistettavien pysäytyssanojen lista on pitkä, niin he uskovat kielen tekstien menettävän alkuperäisen sanoman, kun kaikki pysäytyssanat poistetaan.

Indurkhya ja Damerau (2010) huomioivat morfologisen normalisoinnin tarkoituksen olevan kasvattaa hakutehokkuutta. He sanovat, että sanojen muuttaminen kantasanoiksi voi kasvattaa tiedon löydettävyyttä. Lisäksi he mainitsevat stemmauksen olevan keino muuttaa sanat kantasanoiksi, mutta se on hyödyksi vain, jos stemmaaja typistää sanat oikein. Lemmauksen käyttö mahdollistaisi löydettävyyden olevan varmempaa, mutta voidaan kuitenkin päätellä, että lemmaus on liian raskas operaatio kyseiseen tarkoitukseen.

Indurkhya ja Damerau (2010) kertovat, että painotus on tapa kasvattaa hakusanojen erottuvuutta sekä indeksoinnin joustavuutta. He mainitsevat, että painotusta käytetään, kun asiakirjasta on saatu olennaiset rakenteet talteen, pysäytyssanat on poistettu ja sanat ovat muutettu kantasanoiksi morfologisella normalisoinnilla. Painotuksen toteuttamiseen he esittävät kolmea seuraavaa komponenttia: sanan esiintyvyys (kuvataan tf), asiakirjojen määrä (kuvataan df) ja asiakirjojen pituus (kuvataan n). On olemassa yleinen käsitys, että useimmiten esiintyvät saman kategorian sanat kuvaavat asiakirjan sisältöä (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) huomauttavat, että asiakirjan sisältö voidaan tutkia sanojen esiintyvyyden avulla. Esimerkiksi asiakirjassa voi olla sana ”neuroscience”, joka esiintyy viisi kertaa, jolloin $tf = 5$; Jos sana esiintyisi vain kerran, niin asiakirjan sisältöä ei voitaisi kategorisoida sanan mukaan. Esiintyvyyden ollessa suuri voidaan päätellä teeman olevan johdannainen eniten esiintyvien sanojen kanssa. Jos tiedetään sanan esiintymisen olevan harvinainen asiakirjassa, voidaan antaa ensimmäiselle sanan esiintymiselle enemmän painoarvoa. Indurkhya ja Damerau (2010) esittävät, että tf voidaan muuttaa muun muassa kaavoilla $\log(tf + 1)$ tai $0.5 + 0.5 \times [tf/\max(tf)]$. Viimeisessä kaavassa $\max(tf)$ tarkoittaa suurinta esiintymää mistä tahansa sanasta.

Indurkhya ja Damerau (2010) huomauttavat, että asiakirjojen määrä mittaa kuinka monessa dokumentissa haettu sana esiintyy. He lisäävät, että asiakirjojen määrässä annetaan enemmän painoarvoa harvinaisemmille sanoille, ja vähemmän painoarvoa useammin esiintyville sanoille. He painottavat, että jos sana esiintyy jokaisessa asiakirjassa, niin se ei tuo lisäarvoa haettavasta kohteesta. Painoarvo lasketaan käänteisen asiakirjan esiintyvyyden logaritmista $idf = \log(n/df)$ (Indurkhya & Damerau, 2010). Tällöin, jos sana esiintyy jokaisessa asiakirjassa, niin painoarvoksi tulee 0. Toisaalta, jos sana esiintyy vain yhdessä asiakirjassa niin tämä asiakirja saa suurimman painoarvon. Indurkhya ja Damerau (2010) kertovat, että sanan esiintyvyys yhdistetään usein asiakirjojen määrään kertomalla painoarvot keskenään, sekä ottamalla huomioon asiakirjan pituuden. Tällöin muodostuu kaava $tf \times idf$. He selventävät, että asiakirjan pituudessa on tapana ajatella, että sanan esiintyminen lyhyemmässä asiakirjassa antaa enemmän painoarvoa kuin pidemmässä. Painoarvo lasketaan vertaamalla kyseisen asiakirjan pituutta kaikkien asiakirjojen keskiarvopituuteen $l/avgl$, jossa l on pituus ja $avgl$ on asiakirjojen keskiarvopituus.

3.1.1 Tiedonhakumallit

Indurkhya ja Damerau (2010) esittää kolme erilaista tiedonhakumallia: boolenmalli, vektoriavaruusmalli ja todennäköisyyspohjainen malli. He kuvaavat, että boolenmallia

Taulukko 3.1: Binääri-indeksointi. Muokattu lähteestä (Indurkhya & Damerau, 2010).

	Indeksoitavat avainsanat		
Asiakirja	Tietokone	Peli	Puhelin
A_1	1	0	0
A_2	1	0	1
A_3	0	0	1
A_4	0	1	1

pidetään vanhimpana tiedonhakumallina, jota käytetään enää vain erikoistapauksiin. Vektorimallia ja todennäköisyyspohjaista mallia he pitävät nykyaikaisempina malleina, jotka ovat edelleen käytössä.

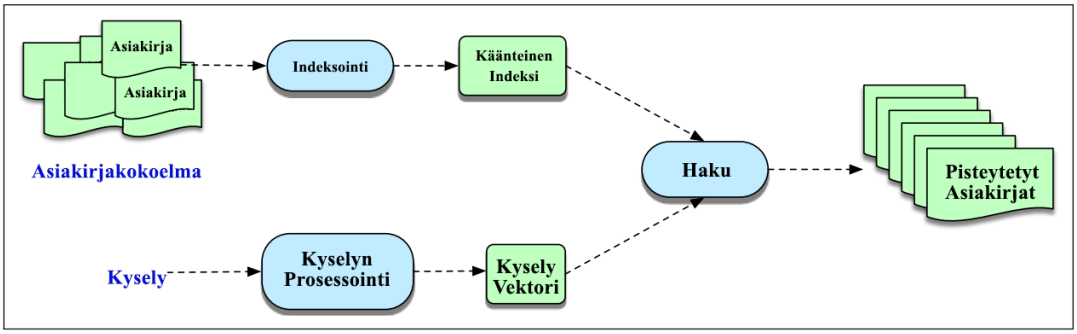
Aiemmin mainituista indeksointitavoista Indurkhya ja Damerau (2010) mainitsevat, että boolenmalli käyttää yleensä manuaalista indeksointia. He tarkentavat, että mallin asiakirjat luokitellaan ennalta määrätyn joukon avainsanoilla. He kertovat, että asiakirjojen hakeminen tapahtuu käyttämällä kolmea Boolen-operaattoria: JA, TAI, sekä JA EI. Indurkhya ja Damerau (2010) muistuttavat hakemisen olevan epäintuitiivista. Jos halutaan hakea sekä tietokonetta että puhelinta, niin ei voida käyttää JA operaattoria, sillä se palauttaa vain asiakirjat, joissa puhutaan kummastakin. Parempana vaihtoehtona ehdotetaan käyttämään TAI-operaattoria. Indurkhya ja Damerau (2010) esittävät, että mallin huonona puolena pidetään sitä, että se ei arvioi asiakirjojen merkittävyyttä. He korostavat, että arviointi voi tapahtua ainoastaan ulkoisten ominaisuuksien kuten julkaisuvuoden ja kirjailijan perusteella. Toiseksi ongelmaksi he esittävät boolenmallin ehdottomuuden. Malli ei voi palauttaa osittain osuvia asiakirjoja, vaikka ne voisivat tuodakin hyvän asiakirjan julki (Indurkhya & Damerau, 2010). Malli noudattaa Boolen-operaattoreita ja ne palauttavat vain tarkasti ehtoihin sopivat asiakirjat.

Indurkhya ja Damerau (2010) huomauttavat, että indeksoitavien asiakirjojen määrän kasvaessa, hakemisesta voi tulla nopeasti hidasta. He kertovat, että tavallisesti voidaan ajatella, että asiakirjat omaavat avainsanat, jolloin asiakirjoja selaamalla valitaan ne, joilla on sopivat avainsanat. Tehokkuuden vuoksi he kertovat, että boolenmalli tallentaa indeksoidut asiakirjat ”käänteiseen” tiedostoon. Käänteisen tiedoston 3.2 idea on tallentaa indeksointitermi, ja linkittää ne asiakirjoihin, joilla on kyseinen indeksointitermi (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) toteavat, että tällöin asiakirjojen hakeminen on nopeampaa kuin käydä jokainen asiakirja läpi ja tarkistaa niiden avainsanat.

Taulukko 3.1 esittää indeksoinnin lähtötilanteen, joka voidaan optimoida käyttämällä taulukon 3.2 periaatetta.

Taulukko 3.2: Käänteinen tiedosto. Muokattu lähteestä (Indurkhya & Damerau, 2010).

Indeksointitermi	Joukko asiakirjojen tunnisteita
Tietokone	A_1, A_2
Peli	A_1, A_2, A_3,
Puhelin	A_3, A_2



Kuva 3.1: Kuvaus ad hoc -tiedonhakujärjestelmästä. Muokattu lähteestä (Martin, 2023).

Kuva 3.1 näyttää käänteisen tiedoston 3.2 sijainnin tiedonhakujärjestelmässä. Käänteinen tiedosto ilmenee kuvassa käänteisenä indeksinä. Kuvasta ilmenee prosessi, jossa kyselyn avulla saadaan haettua käänteisen indeksin kautta kyselyyn sopivat asiakirjat. Saadut asiakirjat ovat pisteytettynä osuvuuden mukaan.

Indurkhya ja Damerau (2010) esittävät, että vektoriavaruusmallit ovat toinen lähestymistapa tiedonhakumalleille. He kertovat vektorimallien kyvykkyydestä hakea asiakirjoja luonnollisen kielen avulla. He huomioivat, että lähestymistapa kuvaa asiakirjat sekä kyselyt vektoreiden avulla. Jokainen indeksointitermi vastaa yhtä vektoriavaruutta, mikä tarkoittaa, että lopullinen ulottuvuuksien määrä on suuri (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) kertovat, että indeksoitavat termit kuvataan yksittäisinä esiintyminä, eli yksittäisen sanan esiintyminen ei vaikuta jonkin toisen sanan esiintymiseen. He esittävät seuraavan kaavan kyselyn ja asiakirjan sovittamiseen:

$$Sim(D_i, Q) = \sum_{j=1}^{p} w_{ij} \cdot w_{qj}$$

$Sim(D_i, Q)$ tarkoittaa asiakirjan D_i samankaltaisuutta kyselyn Q parissa. Samankaltaisuus lasketaan asiakirjavektorin ja kyselyvektorin pistetulona. Pistetulo lasketaan asiakirjan indeksointitermien painoista W_{ij}, verrattuna vastaaviin termeihin kyselyissä W_{qj}. Summa on rajoitettu kyselyn indeksointitermien määrään p. Indurkhya ja Damerau (2010) noteeraavat, että monimutkaisempia kaavoja on olemassa, jotka tarjoavat eroavia tuloksia.

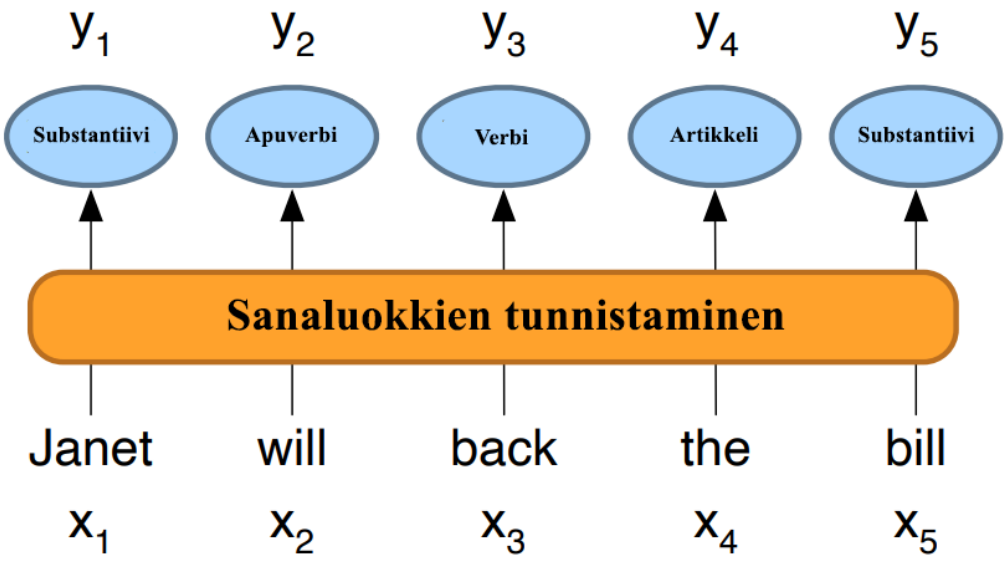
Indurkhya ja Damerau (2010) esittävät todennäköisyyspohjaisten mallien käyttävän luokittelua tiedonhaussa, ja ne perustuvat ajatukseen, että tiedonhaku ei ole aina varmaa. He toteavat, että tämän mallin teoria nojaa todennäköisyysjärjestyksen periaatteeseen (engl. *probability ranking principle*, PRP). PRP kertoo, että paras hakutulos saadaan, kun asiakirjat järjestetään niiden olennaisuuden mukaan (Indurkhya & Damerau, 2010). Manning (2009) täydentää, että kun haemme tietoa, haluamme löytää kaikki olennaiset asiakirjat ennen epäolennaisia. Hakujärjestelmän perustuessa vain asiakirjojen esityksiin eikä itse sisältöön, he mainitsevat, että täydellistä tulosta ei voi taata. He lisäävät PRP:n tarkoituksen olevan auttaa järjestämään tulokset parhaalla mahdollisella tavalla käyttäen todennäköisyyksiä.

Indurkhya ja Damerau (2010) selventävät, että nämä todennäköisyydet lasketaan datan perusteella niin tarkasti kuin mahdollista. He tarkentavat, että järjestelmä pyrkii antamaan parhaat tulokset käytettävissä olevan datan perusteella. Todennäköisyyksien laskemista varten he mainitsevat käytettävän matemaattista Bayesin sääntöä. Sääntö auttaa ennustamaan uusien asiakirjojen kategorioita/piirteitä edellisten asiakirjojen ennusteiden perusteella (Jackson & Moulinier, 2002). Tämä sääntö auttaa valitsemaan ne asiakirjat, jotka ovat todennäköisesti kaikkein hyödyllisimpiä (Indurkhya & Damerau, 2010). Indurkhya ja Damerau (2010) huomauttavat, vaikka todennäköisyyspohjaiset mallit pohjauttavat toimintansa todennäköisyyksiin eivätkä kyselyn ja asiakirjojen samankaltaisuuksiin, on esitetty, että suorituskyky ei ole usein parempi kuin vektoriavaruusmalleissa. Jackson ja Moulinier (2002) lisäävät todennäköisyyspohjaisten mallien toimivan muutoin lähes samoin kuten vektoriavaruusmallit.

3.2 Sanaluokkien tunnistaminen

Sanaluokkien tunnistaminen (engl. *parts-of-speech tagging*) on prosessi, jossa jokaiselle lauseen sanalle annetaan merkintä sen sanaluokasta. Sanaluokkien tunnistaminen asettaa jokaiselle sanalle merkinnän, mihin sanaluokkaan sana kuuluu, esimerkiksi ”run” = verbi. Sanaluokkia voidaan jakaa suljettuihin sanaluokkiin sekä avoimiin sanaluokkiin. Suljettuihin ja avoimiin sanaluokkiin kuuluvat sanat eroavat siinä, kuinka usein niihin lisätään uusia sanoja. Suljetut sanaluokat sisältävät luokat, joihin ei lisätä uusia sanoja tai lisätään harvoin, kuten prepositiot. Avoimiin sanaluokkiin lisätään useasti sanoja, ja sanoja lainataan muualta. (Martin, 2023). Tällaisia luokkia ovat esimerkiksi substantiivit ja verbit.

Parhaimmat sanaluokkien tunnistamisalgoritmit ovat saavuttaneet 97 % tarkkuuden, jota pidetään ihmisten suorituskyvyn kaltaisena (Martin, 2023). Martin (2023) mainitsee



Kuva 3.2: Sanaluokkien kartoitus. Muokattu lähteestä (Martin, 2023).

tutkimuksesta, jossa tarkkuus on pysynyt kyseisessä tarkkuudessa 15 eri kielellä. Algoritmin yleistäminen usealle kielelle suurella tarkkuudella antaa viitteitä onnistuneesta algoritmin toiminnasta. POS-algoritmit kohtaavat haasteita sanojen moniselitteisyyden takia, ja englanninkielisten sanojen sanotaan olevan noin 85–86 % yksiselitteisiä (Martin, 2023). Vaikka jäljelle jäävä määrä on vain 14–15 %, niin sanat, jotka kuuluvat tähän ovat usein esiintyviä sanoja. Martin (2023) muistuttaa, että vaikka sanoilla on useita merkityksiä, niin sanoilla on yksilölliset todennäköisyydet esiintyä tekstissä. Hän mainitsee yksinkertaisen ratkaisun olevan, että valitaan useimmin esiintyvä luokka korpuksesta. Tämä menetelmä perustuu oletukseen, että kontekstin useimmiten esiintyvä merkitys on todennäköisin. Esimerkiksi, englanninkielinen sana ”duck”, joka tarkoittaa ankkaa, sekä kumartumista. Tällöin valitaan joko substantiivi ankka tai verbi kumartua, riippuen kumpaa on esiintynyt enemmän luokittelijan koulutuksessa. Lähtökohta tälle periaatteelle on jo valmiiksi luokiteltu korpus (Martin, 2023). Martin (2023) mainitsee, että luokittelun onnistumisprosentti yksinkertaisella lähestymistavalla on noin 92 %. Haasteena pidetään kuitenkin kieliopillisten rakenteiden monimutkaisuutta ja poikkeuksia, jotka voivat vaikuttaa sanan sanaluokkaan.

3.3 Nimettyjen entiteettien tunnistaminen

Nimettyjä entiteettejä ovat erityiset sanat tai ilmaukset tekstissä, jotka kuvaavat esimerkiksi henkilöiden nimiä, paikkoja, sijainteja ja organisaatioita. Martin (2023) huomauttaa entiteetti-termiä käytettävän usein tilanteissa, joihin termi ei sovi. Entiteetti tarkoittaa

yksikköä tai asiaa, joka on olemassa itsenäisesti, eikä sen olemassaolo riipu muista asioista (Cambridge Dictionary, 2023).

Nimettyjen entiteettien tunnistamista pidetään hyvänä ensiaskeleena luonnollisen kielen käsittelyn tehtävissä (Martin, 2023). Martin (2023) antaa esimerkin, että tunteiden analysoinnissa saatamme esimerkiksi haluta tietää kuluttajan tunteet tiettyä entiteettiä kohtaan. Hän kertoo, että entiteettien tunnistusta pidetään myös ensimmäisenä vaiheena kysymyksiin vastaamisessa tai tekstin linkittämisessä strukturoituihin tietolähteisiin. Hän myös mainitsee, että nimettyjen entiteettien tunnistamista pidetään myös hyödyllisenä tehtävissä, joissa rakennetaan semanttisia esityksiä esimerkiksi tapahtumien erottamisesta.

Jackson ja Moulinier (2002) kuvaavat nimettyjen entiteettien tunnistamista (engl. *named entity recognition*, NER) luokitteluongelmaksi. Ihmisen on helppo luokitella entiteettejä, mutta luokittelun syytä sanoittaa säännöiksi on vaikea ilmaista. Jackson ja Moulinier (2002) kertovat yhtenä ongelmista olevan kaupunkien nimet. Esimerkiksi lause: ”Pietari myy omaisuutensa”, jos lauseen objekti Pietari on kaupunki, niin lause muuttaa merkitystä suuresti. Lauseessa itsessään ei ole mitään erottelijaa, jota voidaan käyttää hyödyksi ja joka kertoisi, onko kyse henkilöstä vai kaupungista (Jackson & Moulinier, 2002). Nimettyjen entiteettien neljä suosituinta tunnistetta ovat **PER**, **ORG**, **LOC** ja **GPE** (Martin, 2023). PER (engl. *people*) kuvastaa ihmisiä ja hahmoja. ORG (engl. *organization*) kuvastaa yhdistyksiä, kuten esimerkiksi yritykset ja urheilujoukkeet. LOC (engl. *location*) maantieteellisiä seutuja, kuten esimerkiksi vuoret ja meret. GPE (engl. *geo-political entity*) kuvastaa maantieteellisiä alueita, kuten esimerkiksi maat ja osavaltiot.

Nimettyjen entiteettien tunnistaminen eroaa sanaluokkien tunnistamisessa, koska sanaluokkien tunnistamisella ei ole segmentointiongelmaa. Tämä tarkoittaa sitä, kun jokaiselle sanalle annetaan yksi tunniste, nimetyn entiteetin tunnistamisen tehtävänä on merkitä tekstiosuuksia, yksittäisten esiintymien sijaan. Tehtävää kutsutaan vaikeaksi osittain segmentoinnin epäselvyyden vuoksi. Segmentoinnissa joudutaan päättämään, mitkä sanat ovat entiteettejä ja mitkä niiden entiteettien rajat ovat. Suurin osa tekstin sanoista ei ole varmuudella nimettyjä entiteettejä. Standardi lähestymistapa kyseisten sekvenssien tunnistamiseen on BIO-tunnistaminen. Tämä menetelmä mahdollistaa käsittelyn sanasta sanaan tapahtuvana sekvenssinä, joka tunnistaa sekä rajat että nimetyn entiteetin tyypin. BIO-tunnistamisessa merkitsemme ensimmäisen sanan, joka aloittaa kiinnostavan alueen, tunnisteella B (engl. beginning of The entity). Merkitsemme alueen sisällä olevat merkit tunnisteella I (engl. inside of the entity) ja kaikki merkit, jotka eivät kuulu mihinkään kiinnostavaan alueeseen, tunnistetaan tunnisteella O (engl. *outside of any entity*). Tunnisteiden määrä on $2n + 1$, missä n kuvaa entiteettien lukumäärää. Kaksi

Taulukko 3.3: Nimettyjen entiteettien tunnistaminen sekvenssi tunnisteilla. Muokattu lähteestä (Martin, 2023).

Sanat	BIO-tunniste	IO-tunniste	BIOES-tunniste
John	B-PER	I-PER	B-PER
Doe	I-PER	I-PER	E-PER
from	O	O	O
the	O	O	O
national	B-ORG	I-ORG	B-ORG
guard	I-ORG	I-ORG	E-ORG
...	...	...	..

vaihtoehtoista tunnistusjärjestelmää ovat IO-tunnistaminen ja BIOES-tunnistaminen. IO-tunnistamisessa menetetään joitakin tietoja poistamalla B-tunniste, eli alkutunniste. BIOES-tunnistamisessa lisätään lopputunniste E potentiaalisen entiteettialueen loppuun. S tunniste tulee alueelle, joka koostuu vain yhdestä sanasta. Suuria eroja menetelmien välillä ei ole, mutta voidaan olettaa IO-tunnistamisen olevan yksinkertaistus, eli ei niin tarkka kuin BIO-tunnistaminen. Lisäksi BIOES-tunnistamisen voidaan olettaa olevan tarkempi, mutta monimutkaisempi kuin BIO- ja IO-tunnistaminen. Taulukko 3.3 kuvaa esitetyt tunnistusjärjestelmät ja antaa kullekin sanalle oman tunnisteen eri tunnistusjärjestelmillä.(Martin, 2023).

Jackson ja Moulinier (2002) jakavat nimettyjen entiteettien tunnistamisen kahteen lähestymistapaan. He kertovat ensimmäiseksi lähestymistavaksi **heuristisen lähestymistavan**, joka pyrkii ratkaisemaan ongelman käyttämällä kieliopillisia ja syntaktisia vihjeitä. He mainitsevat, että aluksi lause luokitellaan korkean luotettavuuden säännöillä. Säännöt koostuvat syntaktisista vihjeistä, jotka voivat syntyä esimerkiksi sanan ympärillä esiintyvistä sanoista ja isoista alkukirjaimista (Jackson & Moulinier, 2002). Esimerkiksi ”Pietari kävelee kadulla.” antaa lisätietoa lauseen objektista, ja se voidaan liputtaa henkilöksi. Jackson ja Moulinier (2002) kertovat, että epäselvissä tilanteissa, jossa esiintyy mahdollisesti paikan- tai henkilön nimi, on tapana valita henkilön nimi. He huomioivat myös, jos maantieteellinen sijainti on lähellä mahdollista paikannimeä, niin luokitellaan se tällöin paikannimeksi. Tällöin myös muut esiintymät käydään muuttamassa vastaamaan luokiteltua entiteettiä. Säännöt ovat sidonnaisia kielen syntaksiin ja suomenkieliset säännöt voivat eritä englanninkielisistä säännöistä (Jackson & Moulinier, 2002).

Toinen lähestymistapa on **tilastoihin perustuva**. Tilastoihin perustuva lähestymistapa hyödyntää Markovin piilomallia. Piilomallin ideana on mallintaa prosessi, joka nimeää entiteetit. Myös tilastoihin perustuva lähestymistapa hyödyntää sääntöjä kuten heuristinen lähestymistapa. Vaikka tilastollinen lähestymistapa perustuu pääasiassa

datan tilastolliseen analyysiin, se voi myös hyödyntää sääntöjä parantamaan entiteettien luokittelua. Eräs kuvaava entiteetin ominaisuus voi olla esimerkiksi, sisältääkö entiteetti numeroita. Toinen voi olla esimerkiksi, että onko sanan alkukirjan iso. Ominaisuudet yksinään eivät riitä entiteetin luokitteluun, mutta ominaisuuksien yhdistäminen entiteetin esiintymisjärjestykseen antaa paremmat mahdollisuudet tunnistaa entiteetti oikein. (Jackson & Moulinier, 2002).

3.4 Entiteettien yhdistäminen

Martin (2023) kertoo entiteettien yhdistämisen tarkoittavan prosessia, jossa tekstissä mainitut entiteetit yhdistetään niitä vastaaviin kohteisiin tietyssä tietokannassa tai ontologiassa. Hän mainitsee, että Wikipedia on yleisimmin käytetty faktoihin perustuva ontologia ja esittelee TAGME:n, suositun menetelmän entiteettien yhdistämiseen käyttäen Wikipediaa tietolähteenään. Wikipedian TAGME-linkkerissä entiteettien joukko määritellään Wikipedian sivujen joukoksi, ja jokaista Wikipedian sivua pidetään ainutlaatuisena entiteettinä e (Martin, 2023). Martin (2023) kuvaa, että algoritmi aloittaa luomalla luettelon kaikista entiteeteistä (Wikipedian sivuista) ja indeksoimalla ne Lucenen (Apache, 2023) kaltaiseen tiedonhakukoneeseen. Hän jatkaa, että se laskee kullekin sivulle sisäisten linkkien määrän ($\text{in}(e)$), joka edustaa kyseiselle sivulle osoittavien muiden Wikipedia-sivujen sisäisten linkkien kokonaismäärää.

Martin (2023) mainitsee, että algoritmi tarvitsee myös ankkurisanakirjan, joka sisältää ankkuritekstit jokaiselle Wikipedian sivulle. Hän tarkentaa, että ankkuritekstit ovat hyperlinkkejä muilla sivuilla, jotka viittaavat tiettyyn Wikipedia-sivuun. Hän kertoo, että ankkurisanakirja luodaan sisällyttämällä sivun otsikko sekä kaikki siihen viittaavat muiden Wikipedian sivujen ankkuritekstit. Algoritmi laskee jokaiselle ankkurijonolle sen kokonaisfrekvenssin ($\text{freq}(a)$), linkkikertojen lukumäärän ($\text{link}(a)$) ja linkkitodennäköisyyden $\text{linkprob}(a) = \text{link}(a)/\text{freq}(a)$ (Martin, 2023). Martin (2023) huomioi, että ankkurisanakirjaa siistitään tarvittava määrä, esimerkiksi poistamalla harvinaisia tai epätodennäköisiä ankkuritekstejä.

Martin (2023) huomioi entiteettien yhdistämisen koostuvan karkeasti esitettynä kahdesta osasta: maininnan havaitsemisesta ja maininnan moniselitteisyyden selventämisestä. Hän kertoo, että **mainintojen havaitsemisessa** TAGME havaitsee maininnat kysymyksen tai linkitettävän tekstin perusteella kysymällä ankkurisanakirjasta jokaista merkkisekvenssiä enintään kuuden sanan osalta. Jaksojen joukko karsitaan heuristiikan perusteella, esimerkiksi poistamalla osajonoja, joiden linkkitodennäköisyys on pieni. Martin (2023) kertoo esimerkkikysymyksen ”When was Ada Lovelace born?” voidaan havaita ankkuri

”Ada Lovelace” ja mahdollisesti ”Ada”, mutta osajonot kuten ”Lovelace”, saatetaan karsia pois alhaisen linkkitodennäköisyyden vuoksi. Sana ”born” ei olisi ankkurisanakirjassa olenkaan, alhaisen linkkitodennäköisyyden vuoksi. Entiteettien yhdistäminen on keskeinen osa informaation hakua ja TAGME tarjoaa tehokkaan tavan yhdistää tekstissä mainitut entiteetit Wikipedian tietoihin.

Martin (2023) kertoo **mainintojen moniselitteisyyden selventämisen** hyödyntävän kahta keinoa, jotka ovat priori-todennäköisyys ja yhteenkuuluvuus/yhtenäisyys. Hän kertoo, että priori-todennäköisyys mittaa todennäköisyyden, jolla sana viittaa tiettyyn entiteettiin, valitsemalla eniten viittauksia olevan entiteetin. Kun taas yhteenkuuluvuuden/yhtenäisyyden avulla TAGME tarkastelee sanan ympäröivää kontekstia, joka auttaa valitsemaan potentiaaliset virheet. Näiden keinojen avulla TAGME suorittaa entiteettien yhdistämisen.

3.5 Tietopohjaiset kysymyksiinvastaamisjärjestelmät

Martin (2023) mainitsee verkossa olevan valtava määrä tekstipohjaista tietoa. Tätä voidaan pitää eräänä kysymyksiinvastaamisjärjestelmien syntyperistä, joka puoltaa tutkimuskysymystä. Martin (2023) lisää, että vaikka suuri määrä tietoa on tallennettu luonnollisena kielenä, niin pitää kuitenkin huomioida, että strukturoidut lomakkeet sisältävät myös arvokasta tietoa. Hän kertoo, että tietopohjaisten kysymyksiinvastaamisjärjestelmien luonteeseen kuuluu luonnollisen kielen kysymyksiin vastaaminen muuntamalla ne sopiviksi kyselyiksi strukturoiduille tietokannoille. Hän lisää myös, että kyseistä lähestymistapaa on pidetty perinteisenä lähestymistapana kysymyksiinvastaamisjärjestelmille varhaisissa sovelluksissa, kuten BASEBALL, joka vastasi luonnollisella kielellä esitettyihin kysymyksiin käyttämällä strukturoitua tietokantaa baseball-peleistä ja tilastoista.

Tietoon perustuvassa kysymysten vastaamisessa on kaksi yleistä suuntausta. Ensimmäinen on graafipohjainen kysymyksenasettelu, jossa tietämyskanta (tietokanta, jonne tehdään kyselyt) sisältää kolmen alkion tietorakenteita, joita kutsutaan RDF-kolmikoiksi. Tietorakenteet koostuvat predikaateista, jotka ilmaisevat relaatiota tai propositiota. Taulukko 3.4 antaa esimerkin RDF kolmikko rakenteesta, jossa subjektina on Ada Lovelace, prediikaattina birth-year sekä objektina vuosi 1815. (Martin, 2023).

Tietopohjaiset kysymyksiinvastaamisjärjestelmät hyödyntävät entiteettien yhdistämistä, jossa tekstimaininnat, kuten ”Ada Lovelace”, liitetään tietopohjan entiteettitunnuksiin.

Taulukko 3.4: Esimerkki RDF kolmikko rakenteesta.

Subjekti	Predikaatti	Objekti
Ada Lovelace	birth-year	1815

Seuraava vaihe RDF kolmikkoa hyödyntävässä kysymykseen vastaamisessa on tunnistaa kysytty erityinen relaatio. Tämä edellyttää lauseiden, kuten ”Milloin syntyi...”, kartoittamista tietopohjan relaatioihin, kuten ”syntymävuosi”. Seuraava esimerkki havainnollistaa, kuinka lauseista muodostuu tiedonhaku: ”Milloin Ada Lovelace syntyi?” $\rightarrow$ syntymävuosi(Ada Lovelace, ?x). ”Mikä on Englannin pääkaupunki?” $\rightarrow$ pääkaupunki(?x, Englanti). Relaation tunnistamiseen sekä linkittämiseen voidaan laskea kysymyksen tekstin ja kaikkien mahdollisten relaatioiden pistetulo. Pistetulon laskeminen antaa tulokset samankaltaisuuksista, joita voidaan käyttää sopivan relaation valitsemiseen. (Martin, 2023).

Molemmat esitetyt tietopohjaiset kysymyksiinvastaamisjärjestelmät sisältävät entiteetin yhdistämistä. Toinen tietopohjainen kysymyksiinvastaamisjärjestelmä perustuu semanttiseen jäsentelyyn, jossa hyödynnetään semanttisen jäsennyksen menetelmiä. Semanttinen jäsentäjä muuttaa kysymyksen tekstipohjaisesta muodosta loogiseen muotoon. (Martin, 2023) Looginen muoto voi olla mikä tahansa muoto, jota voidaan käyttää tekemään kysely tietovarastoon. Kysymyksen looginen muoto on siis joko kyselyn muodossa tai se voi helposti olla myös muunnettavissa sellaiseksi. Martin (2023) huomioi esimerkiksi, että predikaattilaskenta voidaan muuntaa SQL:ksi. Predikaattilaskenta on matemaattisen logiikan osa-alue, joka käsittelee lauseita, joissa on muuttujia (Tieteen termipankki, 2022). Toisin kuin ehdollinen laskenta, joka käsittelee yksinkertaisia lauseita, predikaattilaskenta mahdollistaa monimutkaisempien lauseiden käsittelyn. Esimerkiksi, kun ehdollinen laskenta voi käsitellä lauseita kuten ”Rains” tai ”Does not rain”, predikaattilaskenta voi käsitellä lauseita kuten ”every bird flies” tai ”some students are reading” (Mendelson, 2009). Tällöin esimerkiksi ”every bird flies” voi olla SQL-kyselynä: ”SELECT * FROM Animals WHERE species = ’Bird’ AND verbi = ’Fly’; ”. Tietokantana voi toimia relaatiotietokanta tai jokin muu strukturoitu tietovarasto.

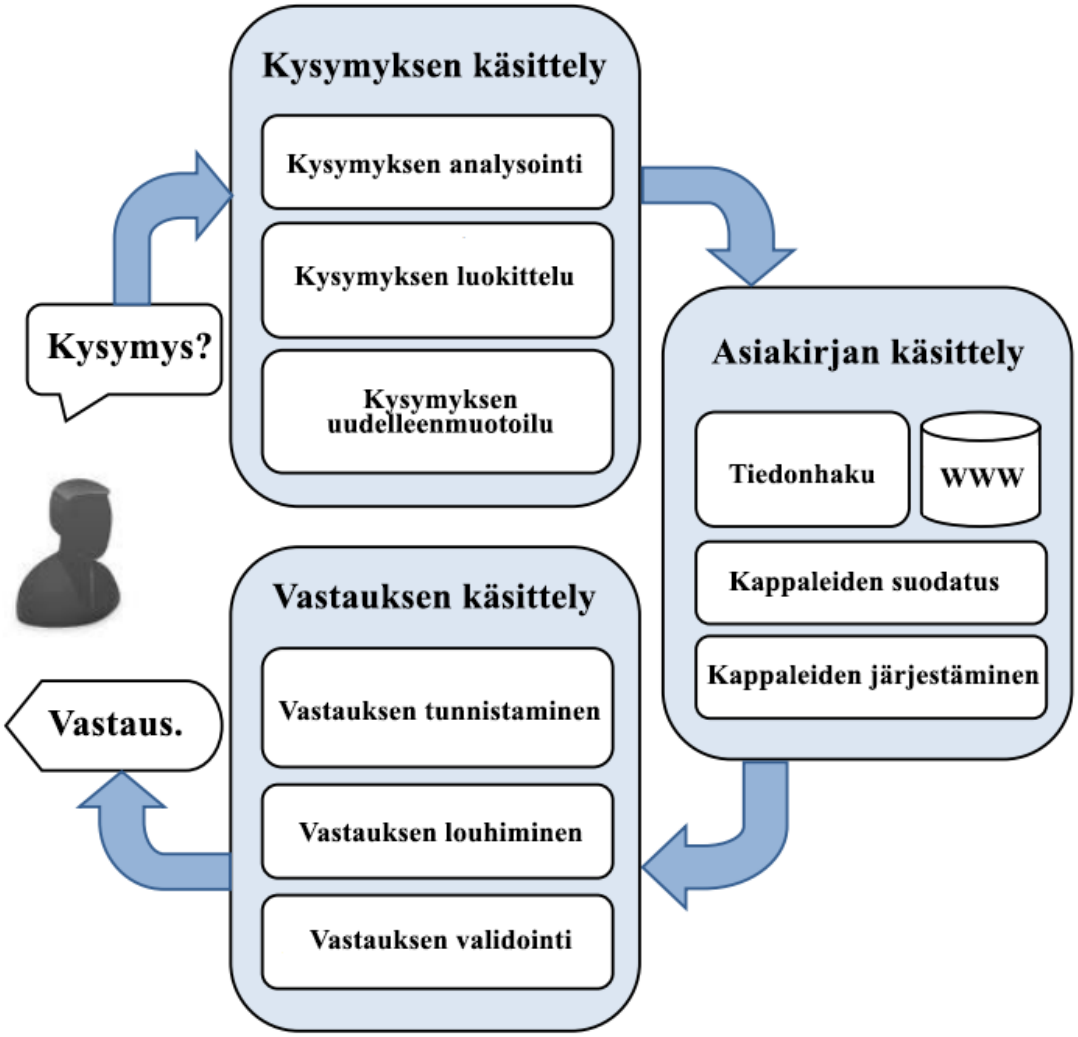
Martin (2023) mainitsee kielimallien olevan vaihtoehto kysymyksiin vastaamiseen, mutta niitä pidetään tulkinnallisesti heikoimpina vaihtoehtoina. Tämä siksi, koska kielimallit toimivat pelkästään annetun tekstin varassa, eikä vastauksia voida perustella antamalla kontekstia vastauksien poluista (Peskoff & Stewart, 2023).

4. Kysymyksiinvastaamisjärjestelmät

Kysymyksiin vastaaminen on tutkimusalue, joka yhdistää tiedonhakujärjestelmät, tiedonlouhinnan sekä luonnollisen kielen käsittelyn (Allam & Haggag, 2012). Hirschman ja Gaizauskas (2001) kertovat kysymyksiinvastaamisjärjestelmien tulevan tarpeelliseksi tiedon määrän lisääntyessä vauhdikkaasti. He mainitsevat tiedonhakujärjestelmien suosion nousseen jo 1960-luvulla. Kuitenkin kysymyksiinvastaamisjärjestelmien suosio on noussut merkittävästi vasta viime vuosina. Tavanomaiseen tapaan olemme tottuneet saamaan hakukoneilta listauksen tuloksista, jotka voivat tarjota vastauksia käyttäjän tarpeisiin. Kysymyksiinvastaamisjärjestelmät ovat edistysaskel tiedon saavutettavuuteen.

Indurkhya ja Damerau (2010) toteavat kysymyksiinvastaamisjärjestelmien olevan haasteellisesti kuvattavia. Intuition tavoin järjestelmää voidaan kuvata sarjana tapahtumia, joissa kysymys tulkitaan ja sen perusteella annetaan vastaus. Ideaalin kysymysten vastaamisjärjestelmän luominen on kuitenkin hyvin laaja tehtävä ja hankala tehtävä saavuttaa (Hirschman & Gaizauskas, 2001). Reaktiona tähän Indurkhya ja Damerau (2010) kertovat, että tilapäisenä ratkaisuna on keskitetty kehitysvoimavarat yksityiskohtaisempiin järjestelmiin, kuten tiedonhakujärjestelmiin (engl. *information retrieval*), luonnollisen kielen käyttöliittymiin tietokannoissa ja suunnata kysymyksiinvastaamisjärjestelmät hyödyntämään hakupohjaisia menetelmiä. Martin (2023) kertoo kysymyksiinvastaamisjärjestelmien pohjautuneen tiedonhakuun ja tietoon 60-luvulla. Hän huomioi useiden kysymyksiinvastaamisjärjestelmien keskittyvän vain faktatietoon, kuten esimerkiksi ”Kuinka löydän asumistukihakemuksen?” Tiedonhakupohjaiset järjestelmät etsivät olennaisen asiakirjan ja pohjauttavat vastauksen asiakirjassa esiintyvään sisältöön. Jijkoun ja de Rijke (2005) mainitsevat, että internetissä esiintyy yleisesti paljon kysymyksiä, mutta ne eivät usein ole faktakysymyksiä. Heidän mukaansa tällaisia ovat esimerkiksi kontekstiin liittyvät selvitys- tai perustelukysymykset. Martin (2023) lisää tietopohjaisten järjestelmien tekevän kyselyitä tietokantoihin, jotka sisältävät aihepiiriin liittyvää faktatietoa.

Martin (2023) jaottelee kysymyksiinvastaamisjärjestelmät kahteen luokkaan niiden tietopohjan perusteella. Hän kertoo avoimen tietopohjan järjestelmien pitävän avoi-



Kuva 4.1: Kysymyksiinvastaamisjärjestelmien kuvaus. Muokattu lähteestä (Allam & Haggag, 2012).

men kontekstin kaikesta, minkä ansiosta voidaan hyödyntää universaalia ontologiaa sekä internetistä saatavaa tietoa. Hän kuvaa suljetun tietopohjan toisaalta vastaavan kysymyksiin yksittäisestä tietoalueesta, kuten verohallinnosta.

Kysymyksiinvastaamisjärjestelmät hyödyntävät kolmea komponenttia, joista jokainen sopii omaksi tutkimusalueekseen. Komponentit mahdollistavat tarvittavat vaiheet kysymyksiinvastaamisjärjestelmän luomiseksi. Kuva 4.1 esittää nämä komponentit, jotka ovat kysymyksen käsittely, asiakirjan käsittely sekä vastauksen käsittely (Allam & Haggag, 2012). Kuva näyttää prosessin tavoin, kuinka kysymyksestä edetään eri komponenttien vaiheiden läpi vastaukseksi.

Seuraavissa aliluvuissa käsitellään kuvan mukaiset komponentit, jotka muodostavat yhteenvedon siitä, miten prosessi etenee kysymyksestä vastaukseksi. Tämän jälkeen esitetään konkreettisia työkaluja kysymyksiin vastaamiseen, kuten Rasan viitekehys.

Toiseksi esitetään Watson-järjestelmä, joka hyödyntää klassisia menetelmiä, sekä TextRunner-tiedonpoimintajärjestelmä. Tiedonpoimintajärjestelmiä voidaan käyttää täydentämään muun muassa kysymyksiinvastaamisjärjestelmiä.

4.1 Kysymysten käsittely

Vastausten muodostamiseksi on ensin ymmärrettävä, mitä kysyjä haluaa. Allam ja Haggag (2012) mainitsevat kysymyksen luokittelu -moduulin olevan ensimmäinen osa, joka rakennetaan kysymyksiinvastaamisjärjestelmiin. He kertovat, että kysymysten luokittelu sisältää kysymyksen kohteen selvittämisen, odotetut vastaustyypit sekä muotoilee kysymyksen useaksi semanttisesti samaa tarkoittavaksi kysymykseksi. He lisäävät kysymyksen muotoilun ja monistamisen voivan vahvistaa tiedonhakujärjestelmien tuloksia sekä niiden laatua. Kysymys voi muodostua dialogin tavoin, eikä konteksti välttämättä hahmotu suoraan yhdestä kysymyksestä. (Hirschman & Gaizauskas, 2001) Dialogien arviointi ei sovellu tämän tutkimuksen laajuuteen, ja siksi tilannetta ei käsitellä tarkemmin. Kysymykset eivät myöskään aina esiinny tavanomaisten kysymysten muodossa. Kysymykset voivat olla esimerkiksi epäsuoria pyyntöjä ja käskyjä (Hirschman & Gaizauskas, 2001). Esimerkiksi käsky: ”tiivistä seuraava”, ei ole suoraan kysymys, mutta ilmaisee halua saada vastaus aiheeseen. Yleisimmät kysymykset ovat faktatietoon pohjautuvia kysymyksiä, mutta Allam ja Haggag (2012) mainitsevat trendin muuttuneen faktatiedosta määrittelykysymyksiin, listauksiin ja miksi-kysymyksiin. Järjestelmien on suositeltavaa ottaa huomioon poikkeavat tavat suorittaa kysymyksiä. Tämä johtuu myös siitä, että järjestelmän käyttäjien tekniset taidot ja luonteet voivat erota toisistaan suuresti (Hirschman & Gaizauskas, 2001).

Hirschman ja Gaizauskas (2001) mukaan kysymykset voidaan jakaa kolmeen eri kategoriaan: tosiasialliset, mielipiteet ja tiivistelmät. Luokittelua voidaan käyttää tunnistamaan kysymysten tyyppi ja reagoida sen mukaan (Hirschman & Gaizauskas, 2001). Esimerkiksi tosiasiallinen tieto vastaa kysymykseen oikein, mielipiteet ovat subjektiivisia ja tiivistelmä sisältää lyhennetyn tekstin oikeasta lähteestä. Allam ja Haggag (2012) esittävät kaksi tarpeellista kohtaa, jotka liittyvät tunnistuksen aiheeseen ja tekevät vastauksen tyypin muodostamisesta helpompaa. He kertovat, että kysymyksestä täytyy selvittää sen tyyppi sekä kohde. Kysymyksen tyypeiksi Allam ja Haggag (2012) listaavat esimerkiksi mikä, miksi, kuka, miten, milloin, missä jne. Allam ja Haggag (2012) antavat esimerkin, jossa kysymys on ”What is the longest river in New South Wales?”. Kyseissä lauseessa kysymystyyppinä on mikä-kysymys, jonka kohteena on ”longest river”. Kun kysymyksen tyyppi ja kohde on selvillä, niin voidaan kerätä kysymyksen avainsanat ja antaa ne tiedonhakujärjestelmälle. Avainsanojen keräämistä varten voidaan hyödyntää

muun muassa nimettyjen entiteettien tunnistamista, pysäytyssanoja sekä sanaluokkien tunnistamista (Allam & Haggag, 2012).

4.2 Asiakirjojen käsittely

Kysymykseen vastaamista varten tarvitaan myös lähde, johon vastaus perustuu. Tämän vaiheen hoitavat tiedonhakujärjestelmät, joiden edeltävänä vaiheena on yleensä kysymysten luokittelu tiedon hakemista varten (Allam & Haggag, 2012). Kysymysten luokitteluvaiheen tärkeys korostuu tiedonhakuvaiheessa. Jos kysymysten luokittelu epäonnistuu, niin tiedonhaku ei onnistu löytämään potentiaalisia vastauksen sisältäviä asiakirjoja (Allam & Haggag, 2012). Allam ja Haggag (2012) muistuttavat, että asiakirjojen käsittelyn vaiheessa ei ole välttämätöntä käyttää vain yhtä tiedonhakujärjestelmää. Usean tiedonhakujärjestelmän yhtäaikainen käyttö mahdollistaa parhaimmat tulokset. Huomiona on myös, että lähes poikkeuksetta yksi tiedonhakujärjestelmän korpuksista on internet. Hirschman ja Gaizauskas (2001) kertovat järjestelmien jakautuvan kolmeen lähdemuotoon: strukturoituun dataan, osittain strukturoituun dataan ja luonnolliseen kieleen. Allam ja Haggag (2012) huomioivat noston tiedonhakujärjestelmistä, jossa tiedonhakujärjestelmät rajaantuvat asiakirjojen palauttamiseen. Tällöin käyttäjän itse pitää arvioida ja valita asiakirjoista oikea oletettu vastaus. Kysymyksiinvastaamisjärjestelmät puolestaan käsittelevät asiakirjat ja valitsevat sieltä sopivan vastauksen (Indurkhya & Damerau, 2010).

4.3 Vastauksen käsittely

Seuraavana vaiheena tiedonhakujärjestelmän jälkeen on vastauksen louhinta (Allam & Haggag, 2012). Allam ja Haggag (2012) huomioivat vastauksen louhinnan olevan vaihe, joka erottaa tavanomaisen tiedonhakujärjestelmän kysymyksiinvastaamisjärjestelmästä. Vastauksien laadut voivat vaihdella suuresti, joten on tärkeää arvioida ja kiinnittää huomiota siihen, millaiset vastaukset ovat soveltuvia. Hirschman ja Gaizauskas (2001) ehdottavat seuraavia kriteerejä tarkasteluun:

- Merkityksellisyys (engl. *relevance*): vastaa kysymykseen.
- Oikeellisuus (engl. *correctness*): vastaus on totuudenmukainen.
- Tarkkuus (engl. *conciseness*): vastaus ei ole liian tiivis eikä liian laaja.

- Täydellisyys (engl. *completeness*): vastaus sisältää vastauksen kaikkiin kysymyksessä ilmenneisiin kohtiin.
- Johdonmukaisuus (engl. *coherence*): vastaus on luettavissa helposti.
- Perustelut (engl. *justification*): vastaus sisältää kontekstin, miksi vastaus valittiin.

4.4 Vastauksen tuottaminen

Luonnollisen kielen tuottamista pidetään alakategoriana tekoälylle sekä laskennalliselle kielitieteelle. Laskennallista kielitiedettä voidaan toisaalta pitää yläkategoriana luonnollisen kielen käsittelylle. Luonnollisen kielen tuottaminen keskittyy ymmärrettävän tekstin luomiseen jollain luonnollisella kielellä, jotta kommunikaatioon soveltuvat kriteerit täyttyvät. Luonnollisen kielen tuottamista voidaan luoda suoraan loppukäyttäjille, tai sitä voidaan pitää välivaiheena, jota ihminen käy muokkaamassa. Kielen tuottamista kuvaillaan mahdollisena keinona parantaa käyttöliittymäkokemusta verkkopalveluissa. (Ell, 2017).

Yhtenä tulkintana kielen tuottamisesta on funktio, johon tarvitaan neljän parametrin joukko (*k, c, u, d*). Parametri *k* tarkoittaa tietolähdettä, *c* on kommunikatiivinen päämäärä, *u* kuvaa kohdeyleisöä ja *d* tuotetun tekstin historiaa. Esimerkkinä tapaus säänkuvaajasta. Säänkuvaajan tietolähteenä käytetään tietokantaa, joka sisältää numeerista dataa eri sääasemilta. Kommunikatiivinen päämäärä on kertoa tietoa kuvauksen kohteena olevasta kuukaudesta. Kohdeyleisön perusteella muokataan generoitava teksti soveltuvaksi yleisön tietotasolle, esimerkiksi iän perusteella. Tuotetun tekstin historiaa pidetään yllä tilanteissa, joissa tekstin luominen on keskustelevan kaltaista. Historian ylläpitämisellä voidaan välttää turhaa toistoa. (Ell, 2017).

4.4.1 Prosessijakauma

Ellin (2017) mukaan luonnollisen kielen tuottamiseen tarvittava prosessi voidaan jakaa seitsemään osaan seuraavanlaisesti:

Esitettävän sisällön rajaaminen (engl. *content determination*). Rajaaminen tapahtuu etsimällä olennainen osa tietolähteestä, esimerkiksi kuukauden perusteella. Rajatusta sisällöstä voidaan koostaa soveltuvat lisäykset tuotettavaan tekstiin. Rajattu sisältö ei välttämättä sisällä soveltuvaa tietoa. Tilanteeseen voidaan reagoida keksimällä oletusarvoinen vastaus, kun haluttua tietoa ei ole saatavilla.

Dokumentin rakentaminen (engl. *document structuring*), eli rakenteen määrittäminen tuotettavaa tekstiä varten. Teksti voidaan rakentaa yhdistämällä yksinkertaisia tiedonpalasia monimutkaisempaan rakenteeseen. Rakenne voidaan kuvitella järjestettyinä aukkojen kokonaisuutena, joihin upotetaan erillisiä lauseita tai tiedonpalasia. Esimerkkinä voidaan ajatella kappale, jonka aloituksessa halutaan kertoa yhteenveto kuukauden säätilanteesta, ja seuraavana kerrotaan omilla virkkeillä poikkeuksellisista tapauksista.

Leksikalisaatiota (engl. *lexicalization*) käytetään tarkentamaan millaisessa muodossa virkkeet ovat. Virkkeet voidaan rakentaa syntaksillisesti eri tavoin, kuten kertomalla yksityiskohtaisesti päivämääriä käyttäen *viikonloppuna sateen olleen 14 %*, tai tietoa tiivistäen: *viikonloppuna satoi hieman*.

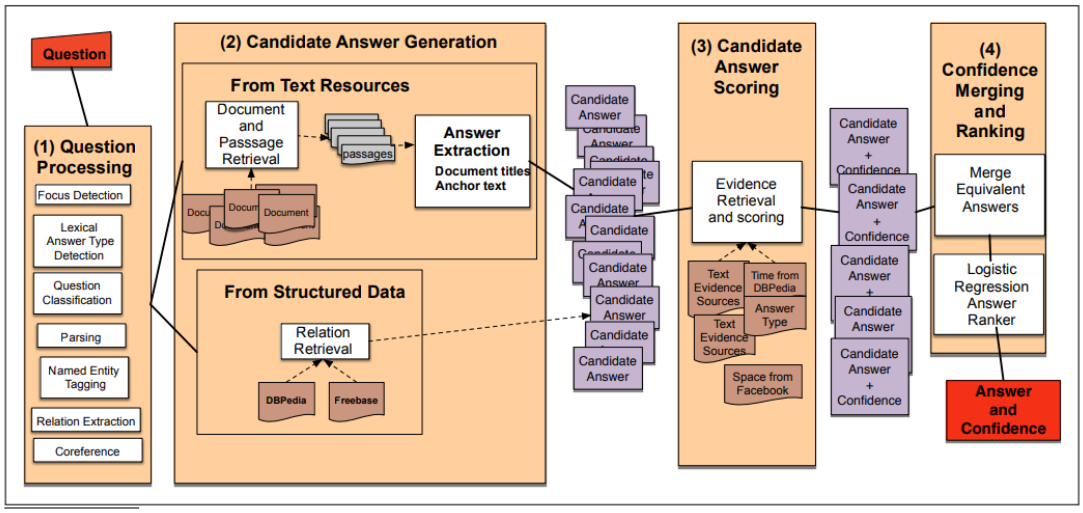
Viittaavan lauseen luominen (engl. *referring expression generation*). Entiteetin toistaminen tuo luonnottomuuden tunnetta. Esimerkiksi kun virkkeen subjekti pysyy samana usean lauseen ajan, niin subjektin toistaminen sen omalla nimellä voi vaikuttaa epäinhimilliseltä.

Aggregointi (engl. *aggregation*) keskittyy lauseiden ja/tai kappaleiden rakenteiden muuttamiseen, riippuen erillisten lauseiden käyttötarkoituksesta. Kahden lyhyen erillisen lauseen voi yhdistää pidemmäksi lauseeksi. Pidemmän lauseen voi myös pilkkoa useammaksi lauseeksi. Aggregoinnin päämäärä on päättää käyttötarkoitukseen sopiva rakenne.

Kielellinen toteuttaminen (engl. *linguistic realization*) huolehtii yksikkö- ja monikkomuotojen oikeellisuudesta, lauseen teeman säilyttämisestä sekä huolehtii tekstin noudattavan aktiivi- ja passiivimuotoja.

Merkintäkielen toteutus (engl. *structure realization*) keskittyy tuotetun tekstin sovittamiseen merkintäkieltä varten. Merkintäkielen sovittaminen tuotettuun tekstiin mahdollistaa sen soveltuvuuden käytettävään palveluun. Palvelu voi olla verkkosivu, jolloin merkintäkielenä voi toimia esimerkiksi HTML. Tuotettu teksti voidaan vaihtoehtoisesti lisätä verkkosivulle DOM:n kautta dynaamisesti.

Teknisestä näkökulmasta luonnollisen kielen tuottaminen suorittaa ainakin seuraavat asiat: sisällön päättäminen/ja tekstin suunnittelu, lauseen suunnittelu ja toteutus. (Reiter & Dale, 1997)



Kuva 4.2: Watsonin arkkitehtuurikuvaus. Muokattu lähteestä (Martin, 2023).

4.5 Klassiset kysymyksiin vastaamisen mallit

Martin (2023) toteaa neuroverkkoja hyödyntävien kysymyksiin vastaamisen mallien olevan uusinta tekniikkaa, mutta klassiset järjestelmät voivat saavuttaa parempia tuloksia. High (2012) informoi Watsonin yhdistävän luonnollisen kielen käsittelyn, hypoteesien luomisen ja arvioinnin sekä dynaamisen oppimisen tarjotakseen tehokkaita, luottamukseen perustuvia ratkaisuja. Hän lisää, että Watson on suunniteltu ymmärtämään suurta määrää strukturoimatonta dataa, joka vuonna 2012 muodosti jopa 80 % prosenttia maailman datasta. Watsonin kyvyt ulottuvat hänen mukaansa myös ihmiskielen monimutkaisuuteen, vivahteikkuuteen, idiomeihin ja moniselitteisyyteen. Martin (2023) analysoi Watsonia, jonka suorituskyky ylsi voittamaan Jeopardy-visailuohjelman haasteen vuonna 2011. Kuva 4.2 esittää Watsonin arkkitehtuurin, joka mahdollistaa kysymyksiin vastaamisen. Kuvan arkkitehtuurista nähdään, että ensiksi kysymys käsitellään sopivaan muotoon. Tämän jälkeen voidaan hakea tietoa, joko rakenteellisesta tietokannasta tai muista tekstiresursseista. Mahdolliset vastaukset tällöin luodaan ja vastaukset pisteytetään. Samankaltaiset vastaukset yhdistetään ja valitaan todennäköisin vastaus.

Martin (2023) kertoo kysymyksen käsittelyn vaiheeseen kuuluvan kysymyksen jäsentäminen, nimettyjen entiteettien tunnistaminen sekä rinnakkaisviittauksien tunnistaminen. Hän mainitsee, että rinnakkaisviittauksien tunnistaminen Jeopardyssa on tärkeää, sillä osa tekstin sisältöä on rinnakkaisviittaus oikealle vastaukselle. Oikean osasisällön löytäminen tapahtuu käsin tehtyjen sääntöjen avulla, esimerkiksi valitsemalla substantiivilauseen määriteosa. Toisena sääntönä voi olla esimerkiksi tilanne, jossa valitaan lauseessa esiintyvä pronomini.

Martin (2023) huomioi, että tekstistä on tärkeää löytää myös leksikaalinen vastaustyyppi. Leksikaalinen vastaustyyppi kertoo vastauksen semantiikasta, joka antaa lisää kontekstia halutulle vastaukselle (Indurkhya & Damerau, 2010). Martin (2023) mainitsee, että kuten rinnakkaisviittaukset löydetään sääntöjen avulla, samoin myös leksikaaliset vastaustyypit tunnistetaan sääntöjen avulla. Hän kertoo Watsonin tapauksessa oletussäännön olevan leksikaalisten vastaustyyppien löytäminen. Oletussääntönä hän kertoo, että valitaan lauseen pääsana, joka on yleensä substantiivi tai verbi. Ferrucci ja muut (2010) kertovat, että vaihtoehtona voidaan ajatella pronominien viittaavaan leksikaaliseen vastaustyyppiin. Sääntöjä voidaan käyttää suoraan antamaan leksikaaliset vastaustyypit tai sääntöjä voidaan käyttää esimerkiksi logistisen regressioluokittimen kanssa (Martin, 2023). Tällöin vastaustyypeille saadaan avuksi myös todennäköisyysprosentti (Ferrucci ja muut, 2010). Lopuksi kysymysluokka selvitetään, eli onko kysymys määrittely, monivalinta, arvoitus vai täytä tyhjät ruudut (Martin, 2023).

Martin (2023) toteaa seuraavaksi vaiheeksi, että kun kysymys on käsitelty, luodaan vaihtoehtoisia vastauksia. Hän kertoo Watsonin käyttävän lähteinään sekä tekstipohjaisia asiakirjoja sekä strukturoituja tietovarastoja. Hän esittelee tekstipohjaisten asiakirjojen käsittelyyn käytettävän yksinkertaistusta hae ja lue -tekniikasta (engl. *retrieve and read*). Hae-osuus etsii olennaiset asiakirjat indeksoidusta kokoelmasta, kun lue-osuus pyrkii käsittelemään haetut asiakirjat ja saaman niistä vastauksen. Tekstistä hakemista varten kyselystä poistetaan pysäytyssanat sekä lisätään painoarvoa sanoille, jotka liittyvät haettuun sanaan. Watson hyödyntää ennalta-arvattavaa Jeopardyn toimintaperiaatetta, jossa useat vastaukset ovat Wikipedian asiakirjojen otsikkoja. Martin (2023) mainitsee, että Watson hyödyntää tekstistä hakemisessa kahta lähestymistapaa. Ensimmäisessä lähestymistavassa etsitään kaikki ankkuritekstit ja toisessa etsitään lauseiden kohdat, jotka ovat lisäksi Wikipedia-asiakirjojen otsikoita.

Seuraavaksi kun potentiaaliset vastaukset on haettu, ne pisteytetään usean eri keinon avuin. Eräs keino on luokittelija, joka luokittelee, voidaanko potentiaalinen vastausvaihtoehto tulkita vastaustyypin alakategoriaksi tai ilmentymäksi (Ferrucci ja muut, 2010). Martin (2023) esittää aiheesta esimerkin, jossa potentiaalisena vastauksena on ”difficulty swallowing” ja vastaustyyppinä on ”manifestation”. Watson tarkastelee vastauksen sekä vastaustyypin ontologiat DBPedia (DBpedia, 2023) sekä WordNet (Fellbaum, Christiane, 2005) palveluista. DBPedian kautta ”difficulty swallowing” voidaan yhdistää ”Dysphagia” entiteettiin. Tällöin saatu entiteetti viedään WordNetiin, ja se palauttaa WordNet tyypin ”symptom”. Sama polku käydään ”manifestation” sanan kanssa ja päädytään sanaan ”condition”. Sanat ”symptom” ja ”condition” muodostavat hyponymian, eli semanttisen suhteen, jossa ensimmäinen termi määrittää toisen termin. Tällöin ehdotetulla vastauksella on potentiaalia olla oikea vastaus. Martin (2023) kertoo, että

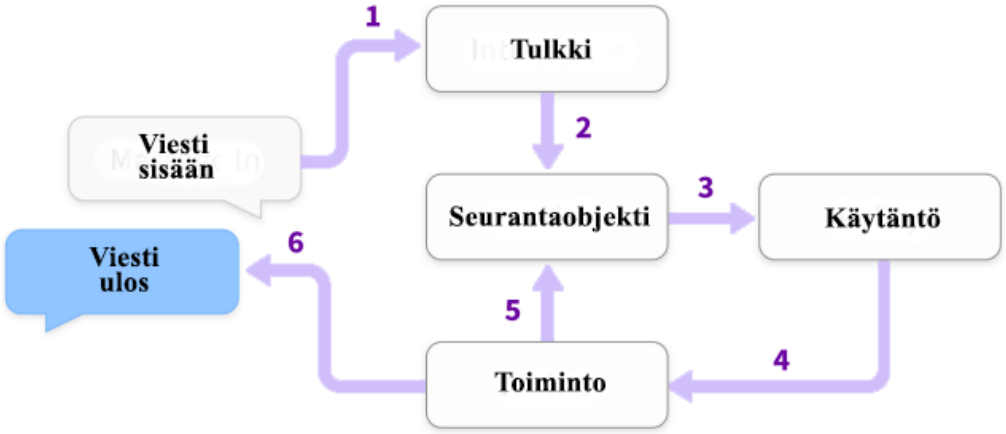
DBpedia mahdollistaa myös ajallisten ominaisuuksien tarkastelemista, joka mahdollistaa tapahtumien kronologisen vertailun. Hän mainitsee, että keino vahvistaa oikeita arvauksia on tehdä käänteinen haku potentiaaliselle vastaukselle ja mitata päällekkäisten sanojen määrä.

Ferrucci ja muut (2010) kuvaavat seuraavaksi vaiheeksi samankaltaisten vastauksien yhdistämisen. He kertovat, että samankaltaisia vastauksia voi syntyä, kun käytetään eri muotoilua tai synonyymeja kuvaamaan samaa asiaa. Heidän mukaansa jokaisella vastauksella on pisteytys ja kun vastaukset yhdistetään, niin myös niiden pisteytykset otetaan huomioon. Kun potentiaalisten vastausten määrä on vähentynyt, voidaan käyttää luokittelijaa arvioimaan ovatko vastaukset oikeita (Martin, 2023). Luokittelija koulutetaan koneoppimisen keinoin ja tuloksena annetaan prosentuaalinen todennäköisyys tulosten oikeellisuuteen (Ferrucci ja muut, 2010). Martin (2023) mainitsee luokittelijan kohtaavan useita vääriä vastauksia koulutuksessa, joten siinä käytetään instanssipainotusta (engl. *instance weighting*) tasapainottamaan epätasapainoista koulutusdataa. Hän kertoo, että instanssipainotus asettaa pienemmän painoarvon jokaiselle väärälle vastausesimerkille. Hän lisää, että luokittelija antaa tulokset perustuen sen varmuusprosenttiin, josta saadaan Watsonin lopullinen vastaus. Prosessi antaa riittävät mahdollisuudet löytää oikea vastaus Jeopardyyn.

4.6 Rasa–vuoropuhelun ja luonnollisen kielen ymmärtämisen viitekehys

Rasa (Rasa, 2023) on avoimeen lähdekoodiin perustuva viitekehys, jonka tarkoituksena on helpottaa koneoppimista hyödyntävän vuoropuhelun hallintaa, sekä luonnollisen kielen ymmärtämistä. Viitekehyksen helppokäyttöisyyden on tarkoitus pienentää tutkimusten ja sovellusten välistä kuilua. Rasa pohjautuu modulaariseen ajatteluun ja koostuu kahdesta päätyökalusta: **Rasa NLU** ja **Rasa Core**. Työkalut toimivat toisistaan itsenäisesti, joka mahdollistaa tarvittaessa työkalujen korvauksen vaihtoehtoisilla ratkaisuilla. Viitekehys hyödyntää tunnettuja pythonin koneoppimiskirjastoja kuten scikit-learn (scikit-learn, 2023), spaCy (SpaCy, 2023) ja Keras (Keras, 2023). Viitekehys tarjoaa myös HTTP rajapinnan, jotta työkaluja voidaan hyödyntää myös muilla ohjelmointikielillä. (Bocklisch, Faulkner, Pawlowski & Nichol, 2017).

Kuvan 4.3 arkkitehtuuri pystyy noudattamaan vuoropuhelua, ja eroaa muista esitetyistä arkkitehtuureista. Erilaiseksi arkkitehtuuriksi tekee muun muassa seurantaobjektikomponentti, joka tallentaa historiatietoa. Historiatiedon ylläpitäminen mahdollistaa keskustelun. Arkkitehtuuria käsitellään tarkemmin myöhemmin.



Kuva 4.3: Rasan arkkitehtuurikuvaus. Muokattu lähteestä (Bocklisch ja muut, 2017).

Sharma ja Joshi (2020) kuvailevat RASA NLU -komponenttia korvana, joka ottaa käyttäjältä vastaan tietoa. Bocklisch ja muut (2017) tarkentavat tämän keskittyvän vain luonnollisen kielen ymmärtämistä vaativiin tehtäviin. He kertovat Rasa NLU:n olevan arkkitehtuurikuvauksessa **tulkki** (engl. *interpreter*). Lisäksi he mainitsevat Rasa NLU:n opetusdatan voivan olla muodoltaan JSON tai merkintäkieli (engl. *markup language*). Bocklisch ja muut (2017) kuvaavat luonnollisen kielen ymmärtämisen komponentin etenevän seuraavanlaisesti:

- Tokenisointi
- Tekstin annotointi spaCy NLP kirjaston avulla
- Tokenit muutetaan vektorimuotoon
- Tekstin vektoroidut tokenit yhdistetään kuvaamaan käsiteltävää tekstiä
- Koulutetaan luokittelija käyttämällä scikit-learn kirjastoa.

Sharma ja Joshi (2020) kuvaavat Rasa Core:n olevan kuin aivot, joka tekee päätöksiä käyttäjän syötteen perusteella. Rasa Core on taas keskustelunhallintaan pohjautuva työkalu, joka koostuu kuvauksessa tulkin jälkeisinä kohtina (Bocklisch ja muut, 2017).

Rasa Core:n osuus koostuu:

Keskustelun ylläpitoon käytetään **seurantaobjektia** (engl. *tracker*), joka tallentaa keskustelun nykytilanteen. Tämä on hyödyllistä, jotta voidaan välttyä toistolta. Tila sidotaan sessiota ylläpitävään objektiin. Tilassa pidetään kirjausta tapahtumista, jotka kuvaavat

keskustelun kulkua, ja samaan tilaan voidaan palata toistamalla nämä tapahtumat. Seurantaobjekti saa tiedon uudesta viestistä ja päivittää tilan vastaavanlaiseksi. (Bocklisch ja muut, 2017).

Rasan **käytäntö** (engl. *policy*) saa tiedon nykyisestä tilasta seurantaobjektilta ja päättää tämän perusteella seuraavan toimenpiteen. Päätös tapahtuu muokkaamalla seurantaobjekti vektorimuotoon, jotta sitä voidaan käyttää luokitteluun. Rasa pohjauttaa toimintaansa luokitteluongelmaan. Viestien ja tilan perusteella voidaan suorittaa luokittelu ennalta määrättyjen toimintojen listaan. Luokitteluun voidaan myös sisällyttää x määrä tiloja. On todettu, että optimaalinen tasapaino säilyy, kun tilojen määrä on vähintään kolme ja enintään kuusi. (Bocklisch ja muut, 2017).

Toimintoihin (engl. *action*) voi kuulua esimerkiksi keskusteluun vastaaminen tai funktion toteuttaminen. Funktio voi olla esimerkiksi suunniteltu tarjoamaan kysymykseen vastaavan osoitteen linkkiä. Toiminnot kuljettavat mukana seurantaobjektia, jotta historiadataa voidaan hyödyntää tarvittaessa. Seurantaobjekti myös päivitetään sisältämään suoritetun toiminnon. (Bocklisch ja muut, 2017).

Riittävä ja laadukkaan koulutusdatan saaminen voi olla hankalaa, siksi on tärkeää kysyä palautetta, kun se on mahdollista. Palautetta voidaan käyttää muodostamaan uutta opetusdataa ja parantamaan sovelluksen toimintaa. (Bocklisch ja muut, 2017). Sharma ja Joshi (2020) kertovat Rasan olevan viitekehyksenä kykeneväinen monimutkaisiin tehtäviin, ilman käyttäjien tarvetta paneutua luonnollisen kielen käsittelyn ytimeen.

4.7 TextRunner-tiedonpoimintajärjestelmä

TextRunner on tiedonpoimintajärjestelmä, jota kutsutaan tarkemmin avoimeksi tiedonpoimintajärjestelmäksi (engl. *open information extraction*). Tavanomaiset tiedonpoimintajärjestelmät keskittyvät kapea-alaisiin pyyntöihin pienistä korpuksista. (Yates ja muut, 2007) Avoimen tiedonpoimintajärjestelmän on toisaalta tarkoitus kerätä kaikki tekstin entiteettien väliset suhteet (Etzioni, Banko, Soderland & Weld, 2008). Yates ja muut (2007) kertovat TextRunnerin louhivan tietoa sadoilta miljoonilta verkkosivuilta, ja tallentavan louhitut tiedot rakenteellisiin monikko-joukkoihin. He myös lisäävät, että rakenteellisten monikkojen poiminta tapahtuu ilman ihmisen antamaa syötettä. Tavanomaista tiedonpoimintaa pidetään luokitteluongelmana ja Etzioni ja muut (2008) noteeraavat suhteista riippumattoman louhinnan olevan huomattavasti vaikeampaa.

Etzioni ja muut (2008) antaa esimerkin vastaavanlaisesta luokitteluongelmasta, jossa annetaan lause S ja suhde R. Esimerkiksi lause ”John works at Microsoft” ja suhde

”works at”. Etzioni ja muut (2008) noteeraavat, että tällöin luokittelijan pitää selvittää antaako lause olettaa, että John työskentelee Microsoftilla. He kertovat, että suhteista riippumattoman louhijan pitää itse selvittää lauseen suhteessa olevan kummatkin entiteetit. He lisäävät, että suhteista riippumaton louhinta tarkoittaa tiedon louhintaa ilman etukäteen hankittua tietoa tekstissä esiintyvistä entiteeteistä.

Etzioni ja muut (2008) kertovat avoimen tiedonpoiminnan tuovan merkittäviä uusia haasteita tiedonpoimintajärjestelmille. He kertovat, että nämä haasteet eroavat tavanomaisista tiedonpoimintajärjestelmistä. Ensimmäisenä näistä haasteista he kuvaavat suhteiden automaattisen poiminnan. Perinteisissä tiedonpoimintajärjestelmissä tämä prosessi vaatii usein käsin merkittyjä syötteitä (Yates ja muut, 2007). Toisin sanoen asiantuntijoiden on manuaalisesti annettava ohjeita järjestelmälle siitä, mitä tietoja etsitään seuraavaksi. Etzioni ja muut (2008) kuvaavat toiseksi merkittäväksi haasteeksi verkossa esiintyvien korpusten heterogeenisyyden. Verkkosisältö kattaa monenlaisia aiheita ja muotoja, jolloin kaikki työkalut, kuten jäsentäjät ja nimettyjen entiteettien tunnistajat, eivät välttämättä toimi aina yhtä tarkasti (Yates ja muut, 2007). Etzioni ja muut (2008) noteeraavat tämän johtuvan siitä, että korpus, jolla näitä työkaluja koulutetaan, eroaa usein todellisesta verkkosisällöstä. He kuvaavat myös kolmannen haasteen liittyvän järjestelmän skaalautuvuuteen ja tehokkuuteen. Avoimien tiedonpoimintajärjestelmien on pystyttävä käsittelemään valtavia dokumenttikokoelmia tehokkaasti, mikä tarkoittaa usein, että ne on rajoitettu tekemään vain yksi nopea läpikäynti datasta (Yates ja muut, 2007).

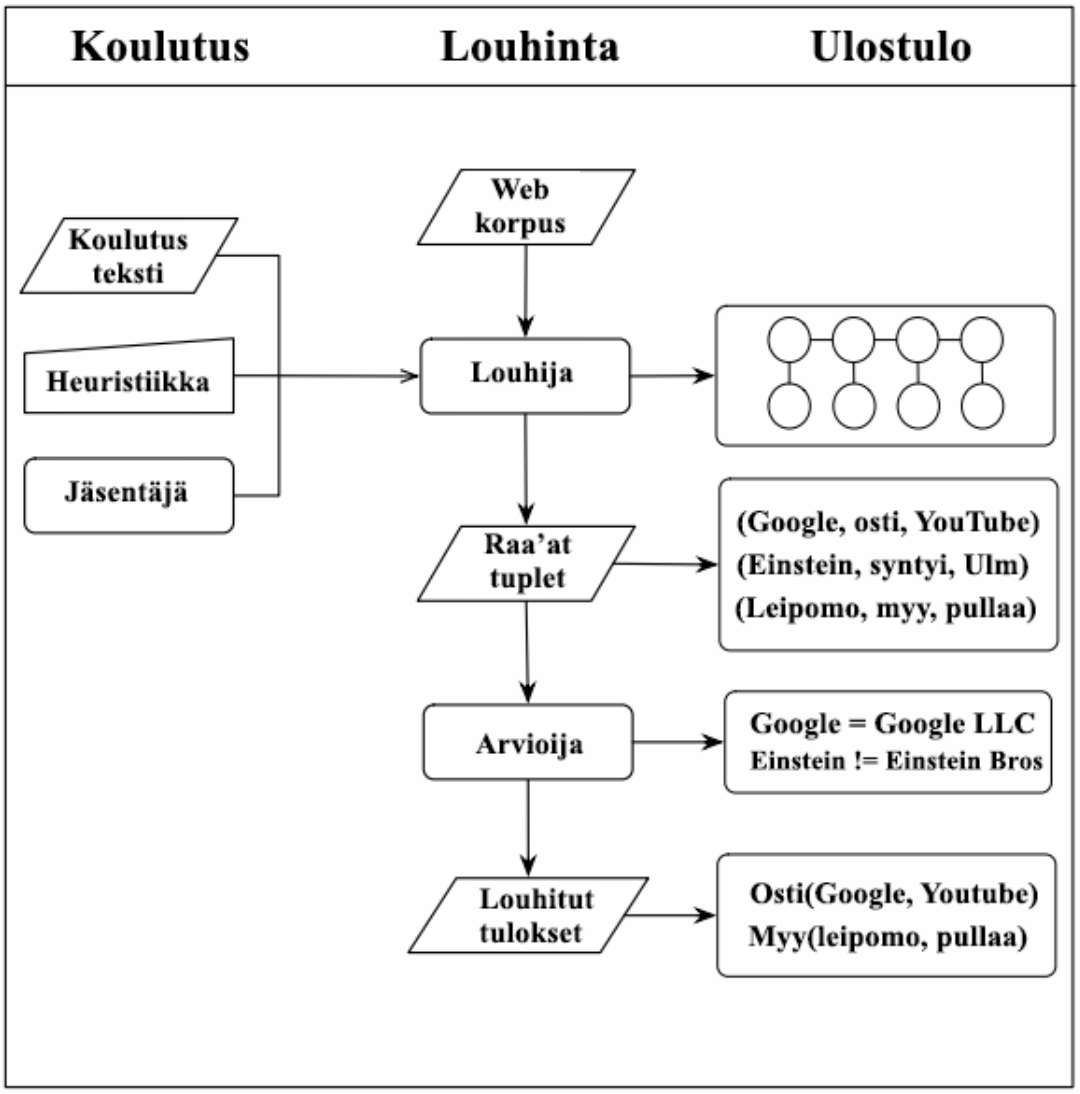
Yates ja muut (2007) kertovat, että seuraavat TextRunnerin komponentit ovat tehty ratkaisemaan edellä mainittuja haasteita:

Yksittäinen läpikäynti -poimija: TextRunnerin poimija käy läpi kaikki asiakirjat yhdellä kerralla, merkiten lauseet sanaluokka-tageilla ja substantiivilausekkeilla. Se arvioi jokaisen substantiivilausekkeen parin etäisyyden ja päättää suhteen poiminnasta useiden rajoitteiden perusteella.

Itseohjattu luokittelija: Vaikka koko verkon täydellinen jäsentäminen ei ole tehokasta, TextRunner käyttää jäsentäjää luodakseen koulutusesimerkkejä poimintaan. Tämä luokittelija voi päättää, onko POS-merkittyjen sanojen sekvenssi oikea poiminta.

Synonyymien resoluutio: TextRunner voi poimia useita merkkijonoja, jotka edustavat samaa suhdetta. Resolver-järjestelmä ryhmittelee TextRunnerin poiminnat luodakseen synonyymisten entiteettien ja suhteiden joukkoja.

Kyselyliittymä: TextRunner rakentaa käänteisen indeksin poimituista monikoista, mahdollistaen nopeat, interaktiiviset ja tehokkaat relaatiokyselyt.



Kuva 4.4: Kuvaus TextRunnerin arkkitehtuurista. Muokattu lähteestä (Etzioni ja muut, 2008).

TextRunnerin arkkitehtuuri muodostuu kuvassa 4.4 seuraavanlaisesti. Koulutus-sarake viittaa TextRunnerin koneoppimisen komponenttiin, jota hyödyntää myös järjestelmän louhija-komponentti. Louhijan avulla saatu tieto voidaan tallentaa monikkoihin, jotka voidaan jälkikäsitellä. Jälkikäsittelystä saadaan lopulliset tulokset, jotka voidaan kuvata esimerkiksi predikaattilogiikan tavoin. Arkkitehtuuria kuvataan tarkemmin moduulien eri kuvauksissa.

Järjestelmän arkkitehtuuri rakentuu neljästä moduulista:

Etzioni ja muut (2008) kertovat oppija-moduulin (engl. *learner*) koostuvan itseohjatusta koneoppimisen algoritmista. He tarkentavat, että itsevalvottu tarkoittaa, että algoritmi korjaa itse algoritmimallin koulutukseen tarkoitetut todellisen tilan merkit, jotta niitä ei tarvitse manuaalisesti asetella. Merkit ovat sidottuja yksittäisiin dataentiteetteihin, ja ne

ilmoittavat niiden todellisen tilan, joka on koneoppimisen algoritmien tavoiteltu tila.

Etzioni ja muut (2008) mainitsevat TextRunnerin tarkastelevan lauseiden joukkoa ja sen pyrkivän löytää lauseiden joukosta entiteettien välisiä suhteita. TextRunner voi esimerkiksi tarkastella lausetta ”John hit a ball” ja ymmärtää, että sanojen ”John” ja ”ball” välillä on suhde. Tämän jälkeen, kun TextRunner on tunnistanut mahdolliset suhteet, on sääntöjen avulla selvitettävä, ovatko nämä suhteet oikeita vai eivät (Etzioni ja muut, 2008). Sääntö voi olla esimerkiksi sellainen, jonka mukaan, jos lauseessa on subjekti, verbi ja objekti, niin kyseessä on todennäköisesti pätevä suhde. Etzioni ja muut (2008) mainitsevat, että vaikka parhaimmat sääntöjen tekijät pääsivät lähelle koneoppimisen algoritmeja, niin ihmisten tekemät säännöt eivät olleet tarpeeksi kattavia kaikkiin tilanteisiin. Huomio pätee myös yleiseen tilanteeseen, jossa asiantuntijajärjestelmät vaativat valtavan määrän työtä ylläpitää järjestelmää sekä ammattitaitoa. Tilannetta kutsutaan tietämystekniikan pullonkaulaksi (engl. *knowledge engineering bottleneck*).

TextRunnerin oppijamoduuli on koulutettu monikoilla, joissa entiteettien välinen suhde voi olla mikä tahansa (Yates ja muut, 2007). Etzioni ja muut (2008) huomioivat koulutusdatan olevan liian suuri, jotta datalle voidaan asettaa merkit käsin. Tähän ratkaisuna he pitävät jäsentäjiä, jotka käsittelevät riippuvuussuhteita. Martin (2023) kertoo riippuvuuden jäsentäjien voivan muodostaa suunnatun verkon, joka yhdistää kaikki lauseen sanat. Etzioni ja muut (2008) lisäävät, että suunnattu verkko antaa tietoa sanojen välisistä riippuvuuksista ja mahdollistaa myös monikkojen louhimisen.

Lisämerkintänä Etzioni ja muut (2008) esittävät hypoteesin, jossa jäsentäjien käyttö suoraan internetissä esiintyvään sisältöön ei ole järkevää. Internetin luonteen vuoksi sisältö on hyvin jakautunutta eri aihepiirien välillä, ja saatava tieto on rakenteetonta sekä voi esiintyä monella eri tapaa kuvattuna.

Etzioni ja muut (2008) kertovat toisen moduulin olevan louhija (engl. *extractor*), jota pidetään järjestelmän keskeisenä osana. He kertovat, että louhija tunnistaa ja hakee olennaisia tietoja tunnistettujen entiteettien välisten suhteiden perusteella. Lisäksi he mainitsivat ihmisten tarkistaneen satunnaisotannan louhijan tuloksista, ja saaneen 92,9 % oikeellisuusprosentin.

Kolmas moduuli on arvioija (engl. *assessor*), joka tunnistaa samaa asiaa kuvaavat suhteet ja objektit, jotta voidaan välttyä toistolta. Arvioija ensin normalisoi louhitut entiteettien ja suhteiden nimet. Seuraavaksi arvioija vähentää turhaa tietoa, yksinkertaistamalla lauseita, esimerkiksi poistamalla lauseesta adjektiiveja. Turhan tiedon poistamisen vaihe voi usein hävittää myös tärkeää tietoa, jolloin sanojen ja lauseiden semantiikka voi muuttua. Moduuli hyödyntää tämän tekemiseen ohjaamatonta koneoppimisen algoritmia. Kyselyn prosessoija on neljäs moduuli, jonka tarkoitus on indeksoida TextRunnerin lopputulos.

Indeksointi suoritetaan jakautuneesti eri palvelimille tai vastaaville arkkitehtuureille, joka mahdollistaa faktojen tehokkaan läpikäymisen. (Etzioni ja muut, 2008).

5. Suomen kielen sovittaminen luonnollisen kielen käsittelyyn

Korenius ja muut (2004) mainitsee suomen kielen olevan tunnettu rikkaasta morfologiastaan, joka koostuu suuresta määrästä taivutus- ja johdannaismuotoja. Tämä monimutkaisuus tuo haasteita erityisesti stemmauksen ja lemmauksen kaltaisiin prosesseihin (Korenius ja muut, 2004). Suomen kielen sanamuotojen variaatiot ovat moninaisempia verrattuna esimerkiksi muihin eurooppalaisiin kieliin (Ursin, 2002). Martin (2023) huomioi sanaluokkien tunnistamisalgoritmien vaativan muutoksia morfologisesti rikkaille kielille. Morfologisesti rikkaita kieliä ovat muun muassa Unkari, Turkki, Viro ja Sloveeni (Martin, 2023). Hämäläinen ja Alnajjar (2021) huomauttavat uraalisten kielien jakavan morfologisesti rikkaat ominaisuudet. Voidaan tehdä oletus, että suomen kieli on morfologisesti rikas. Rikkaat ominaisuudet tekevät kielien käsittelemisestä haasteellisempaa kuin esimerkiksi englannin kielen käsittelemisestä. Tekstiä voidaan esikäsitellä samankaltaisella prosessilla kuten englannin kielessä. Asiakirjoja voidaan hakea muun muassa uutisista, verkkofoorumeilta sekä louhimalla tietoa erilaisista internetin sijainneista (Virtanen ja muut, 2019). Sekalainen tekstidata jäsennetään lauseiksi, tokenisoidaan sekä parsitaan (Virtanen ja muut, 2019)

Seuraavana esitellään konkreettisia suomen kielen haasteita sekä ongelmia. Tämän jälkeen esitellään suomen kielen edistyksiä luonnollisen kielen käsittelyjen työkaluissa.

5.1 Suomen kielen haasteet ja ongelmat

Suomen kielessä sanojen yhdistäminen johtaa ongelmiin hakutulosten parissa. Jos halutaan löytää esimerkiksi yhdyssanojen toinen tai jälkimmäinen sana kirjoittamatta niitä täysin oikein, niin hakutuloksesta ei tule halutunlainen. Yhdyssanat ovat yleinen ongelmien tuottaja ja joskus voidaan tehdä päätös, että halutaan indeksoida yhdyssanat erikseen. Tämä helpottaa tiedonhakujärjestelmiä hakemaan eri variaatioita mahdollisesta

yhdyssanasta. Yhdyssanojen ongelmat korostuvat muun muassa lemmatisoinnissa. Korenius ja muut (2004) mainitsevat erään suomen kielen ryhmittelysovelluksen toimivan paremmin lemmatisoidulla kuin stemmatulla tekstillä. He lisäävät suomenkielisessä sanakirjassa olevan 200 000 esiintymää ja kahden kolmasosan niistä olevan yhdyssanoja. TurkuNLP on kehittänyt työkaluja, jotka ovat suunniteltu erityisesti suomen kielen monimutkaisen morfologian käsittelyyn (Luotolahti, Kanerva, Laippala, Pyysalo & Ginter, 2015).

Suomenkieliset nimeämiskäytännöt tuovat haasteita nimien tunnistamiseen. Suomen kielessä ei ole tapana laittaa kaikkia nimeen liittyviä osia isolla alkukirjaimella, toisin kuin englannin kielessä. Kyseinen eroavaisuus hankaloittaa yksinkertaisen heuristiikan muodostamista nimien erottamiseen. (Luoma, Oinonen, Pyykönen, Laippala & Pyysalo, 2020) Luoma ja muut (2020) mainitsevat toisen haasteen, joka ilmenee lainausmerkkien sisällä. Lainausmerkit voivat sisältää mainoslausahduksen tai kuvauksen tuotteesta. Kyseisten epäselvyyksien selvittämiseen voidaan käyttää olemassa olevien mainoslauseiden tarkastamista mahdollisesti olemassa olevasta listauksesta (Luoma ja muut, 2020). Luoma ja muut (2020) pitävät kyseistä tilannetta nimettyjen entiteettien tunnistamiseen haasteellisena, mutta sen ratkaisun periaatetta voidaan hyödyntää tavanomaisissa luonnollisen käsittelyn sovelluksissa. He ehdottavat, että tavanomaiset sovellukset voivat keskittyä tiettyyn kontekstiin tai tietyn aiheen asiasanastoon, jolloin usein esiintyvät epäselvät tilanteet voidaan koota omaksi listaukseksi. Toiseksi haasteeksi he mainitsevat, kun kirjallisuudessa puhutaan elottomista esineistä elollisina, esimerkiksi ajoneuvoja voidaan kutsua ihmisen nimellä ja viitata, että tämä teki jotain elollista.

Koska suomen kielen monimutkainen taivutusmuotojärjestelmä asettaa erityishaasteita, Brunni, Lehto, Jantunen ja Airaksinen (2015) korostavat lemmatisoinnin erityistä tärkeyttä. He toteavat myös, että rikas morfologia tekee kielen analysoinnista haastavaa, kun suomen kielen morfeemit voivat esiintyä monimutkaisina yhdistelminä. Käyttäjien tuottama sisältö, erityisesti sosiaalisen median alustoilla, sisältää usein epävirallista kieltä, slangia ja lyhenteitä, jotka poikkeavat perinteisestä kirjakielestä (Lourentzou, Manghnani & Zhai, 2019). Partanen, Hämäläinen ja Alnajjar (2019) kertovat, että internetissä esiintyvät tiedonlähteet kuten sosiaalinen media ja erilaiset blogit voivat sisältää puhekielistä suomea. He mainitsevat, että puhekielen ja tavallisen kirjoitetun kielen eroavaisuus voi vaikuttaa negatiivisella tavalla luonnollisen kielen käsittelyn sovelluksiin.

Wilcock (2001) esittää XML:ään pohjautuvia keinoja toteuttaa luonnollisen kielen tuottamista. Hän kertoo, että kyseiset keinot ovat mahdollisia tuottaa Javalla ja perustuen avoimeen lähdekoodiin, jotta ne ovat avoimesti saatavilla. Lisäksi hän mainitsee

luonnollisen kielen tuottamisen XSL:ää (engl *eXtensible Stylesheet Language*) käyttämällä olevan suomen kielelle haasteellista. Morfologisen rikkauden vuoksi XSL ei välttämättä tarjoa tarvittavia ominaisuuksia tai joustavuutta kielen käsittelyyn (Wilcock, 2001). Korenius ja muut (2004) huomioivat suomen kielen sanojen vievän huomattavasti suuremman tallennustilan kuin englannin kielen sanat. Tämä johtuu muun muassa siitä, että suomen kielessä esiintyy useita sanojen taivutuspäätteitä. Taivutuspäätteet voivat esiintyä myös kerroksittain, joka kasvattaa taivutettujen sanojen määrää huomattavasti.

5.2 Edistyksiä luonnollisen kielen käsittelyn ratkaisuissa

Suomen kieli on hyvin tuettu kaupallisissa sovelluksissa, kertovat Hämäläinen ja Alnajjar (2021). Avoimeen lähdekoodin sovelluksiin Hämäläinen ja Alnajjar (2021) suosittelevat Voikkoa (Voikko, 2023). He korostavat puhutun suomen kielen eroavan kirjoitetusta, ja että sille on tehty oma Python-kirjasto (Murre, 2023), joka kääntää puhekielen kirjakieleen.

Sanaluokkien tunnistamista varten suomen kielelle on korpuksia, joita voidaan käyttää arvioimaan tunnistuksen tarkkuutta. Virtanen ja muut (2019) arvioivat BERT-mallin suoritusta sanaluokkien tunnistuksessa ”the Turku Dependency Treebank”(TDT), ”FinnTreeBank”(FTB) ja ”Parellel UD treebank”(PUD) tarjoamien kokoelmien avulla. BERT (engl. *bidirectional encoder representations from transformers*) eroaa tavallisista kielimalleista siinä, että sen tiedonkulku on kaksisuuntainen, kun taas perinteiset mallit toimivat joko vasemmalta oikealle tai toisinpäin (Devlin, Chang, Lee & Toutanova, 2018). Englanninkielisen nimen mukaan tiedon kulku on molemmin puolin suuntaavaa, ja tämä mahdollistaa tehtävien kuten kysymyksiin vastaamisen, nimettyjen entiteettien tunnistamisen ja tekstin luokittelun korkealla tarkkuudella (Devlin ja muut, 2018).

Nimettyjen entiteettien tunnistamista varten oli olemassa vain yksi suomenkielinen korpus (Virtanen ja muut, 2019). TurkuNLP on tehnyt tieteellisen julkaisun Virtanen ja muut (2019) jälkeen suomenkielisen nimettyjen entiteettien tunnistajan. Nimettyjen entiteettien tunnistaja on mahdollista ajaa palvelinympäristössä Python-ohjelmointikielellä (Turku NLP, 2020). Entiteetintunnistaja noudattaa OntoNotes NE tyyppejä, eli entiteetit luokitellaan ennalta määrättyihin kategorioihin, joita esitellään tarkemmin taulukossa 5.1.

Hämäläinen ja Alnajjar (2021) mainitsee Omorfin olevan yksi parhaiten tuetuista työkaluista morfologiseen analyysiin. Se perustuu äärellisen tilan muuntajaan ja toimii yhdessä rajoitekielioppiin perustuvien lauseiden todellisen merkitysten selvittäjien ja

Taulukko 5.1: OntoNotesin nimettyjen entiteettien tyypit. Käännetty osittain sivulta: `https://turkunlp.org/fin-ner.html`

PERSON	Ihmiset, myös fiktiiviset
NORP	Kansallisuudet tai uskonnolliset tai poliittiset ryhmät
FAC	Rakennukset, lentokentät, valtatiet, sillat jne.
ORG	Yritykset, virastot, laitokset jne.
GPE	Maat, kaupungit, osavaltiot
LOC	Muut kuin GPE-paikat, vuoristot, vesistöt.
PRODUCT	Esineet, ajoneuvot, elintarvikkeet jne. (Ei palveluja.)
EVENT	Nimetyt hurrikaanit, taistelut, sodat, urheilutapahtumat jne.
WORK OF ART	Kirjojen, laulujen jne. otsikot.
LAW	Lainsäädännöksi muutetut nimetyt asiakirjat
LANGUAGE	Mikä tahansa nimetty kieli
DATE	Absoluuttiset tai suhteelliset päivämäärät tai ajanjaksot
TIME	Päivää pienemmät ajat

syntaattisten jäsentäjien kanssa. Äärellinen tilan muuntaja on muunnos äärellisestä tilakoneesta, joka ottaa syötteeksi merkkijonon, ja tilojen siirtymien avulla tuottaa tulostusmerkkijonon (Mohri, 1997). Muuntajaa voidaan esimerkiksi käyttää muokkaamaan sanan yksikkömuoto ”cat” monikkoon ”cats”. Rajoitekielioppi puolestaan on formalismi kieliopista, jota käytetään jäsentämään lauseiden todellisia merkityksiä (Karlsson, 1990). Toisena vaihtoehtona Hämäläinen ja Alnajjar (2021) mainitsevat FinnPos lemmatisoijan sekä morfologisen jäsentäjän, joka pohjautuu ehdollisiin satunnaisiin kenttiin (engl. *conditional random field*). Ehdollinen satunnainen kenttä on tilastollinen mallinnuskeino koneoppimisessa, jota voidaan käyttää esimerkiksi sanaluokkien tunnistamista varten (McCallum & Li, 2003).

Luonnollisen kielen tuottamiseen on työkaluja kuten Omorfi ja Syntax Maker. Syntax Maker on Omorfista riippuvainen työkalu, jonka tarkoituksena on luoda syntaksillisesti oikeita lauseita lemmoista ja niiden suhteista. Työkalu kehitettiin osaksi runokonetta, jolla luotiin runoutta suomen kielellä (Hämäläinen & Rueter, 2018). Hämäläinen ja Alnajjar (2021) noteeraavat Helsingissä olevan vahva keskittyminen laskennalliseen luovuuteen. He kertovat, että suomen kielelle on luotu runontuottajia, kuten Keinoleino ja Poeticus. Lisäksi he mainitsevat, että on olemassa työkaluja myös uutisotsikoiden parantamiseen ja kokonaisten uutisartikkeleiden automaattiseen tuottamiseen.

Vektoriesitykset ovat yleistyneet luonnollisen kielen käsittelyssä ja Hämäläinen ja Alnajjar (2021) mainitsevat, että suomen kieli ei ole poikkeus. Suomen kielelle on esimerkiksi saatavilla esikoulutettuja word2vec- ja fastText-malleja. BERT-malleja on myös saatavilla monikielisenä versiona tai erikseen suomeksi koulutettuna (Virtanen ja muut, 2019). Lisäksi on olemassa SemFi-malli, joka on relaatiotietokanta, joka tallentaa sanojen semanttisia suhteita niiden syntaktisten yhteisesiintymisten perusteella (Hämäläinen & Alnajjar, 2021). Hämäläinen (2018) kertoo, että kantaan tallentuu tieto siitä, mikä auttaa tunnistamaan lauseiden välisten jäsenten semantiikan. Hän lisää, että tietokannassa on tällöin taulu sanoille ja niiden suhteille, jotka ovat yhdistettynä viiteavaimilla.

Suomen puheentunnistus on myös tuettu useilla eri työkaluilla. Hämäläinen ja Alnajjar (2021) mainitsevat eräillä festivaaleilla olleen käytössä suomalainen ääni nimeltään Suopuhe. Toisena puhetta tuottavana sovelluksena mainitaan olevan eSpeak-ng, joka tukee suomen kieltä (Hämäläinen & Alnajjar, 2021). Hämäläinen ja Alnajjar (2021) havainnoivat luonnollisen kielen käsittelyn suomen kielen tutkimuksessa olevan suurimmaksi osaksi avointa. He pitävät tätä kriittisenä kielille, jotka ovat eivät ole puhutuimpien kielien joukossa. He mainitsevat, että jos suomen tutkijat eivät julkaisisi löydöksiään julkiseksi, niin monet muut eivät pystyisi tuottamaan korkeatasoisia työkaluja.

6. Yhteenveto

Usein kysyttyjen kysymysten vastauksien hankkiminen on mahdollista kysymyksiinvastaamisjärjestelmillä. Kysymyksiinvastaamisjärjestelmät koostuvat monesta erillisestä komponentista ja niitä voidaan hioa monella eri tavalla. Usein päädytään kuitenkin kompromissiin esimerkiksi suorituskyvyn vaatimuksissa tai tilavaatimuksissa. Riippuen siitä, kuinka kompromissi tehdään, voidaan päätyä saamaan tarkempia tuloksia suuremmalla suorituskyvyllä ja/tai tilavaativuudella. Päätökset esiprosessoinnista lähtien vaikuttavat kumuloituvasti kysymyksiinvastaamisjärjestelmän lopputulokseen. Esimerkkinä, jos tekstin esiprosessointi hoidetaan huolimattomasti, niin samaa tarkoittavien sanojen sekä merkityksettömien merkkien määrä hidastaa käsittelyä. Ylikehitetyt lemmatisoijat voivat tuottaa hyviä tuloksia, mutta suhteutettuna usein kysyttyjen kysymysten vastauksien antamaan järjestelmään, järjestelmä voi olla liian tehoton. Tärkeää on pitää mielessä mittakaava ja riittävä tulosten tarkkuus. Kun järjestelmän vaatimukset ovat otettu huomioon, voidaan rakentaa kysymyksiinvastaamisjärjestelmä, joka ei ole ylikehitetty, sekä on riittävän tarkka ja suorituskykyinen.

Luonnollisen kielen käsittelyn luku 2 toimii tutkielman perustana, sillä siinä esitellään luonnollisen kielen käsittelyä sekä sen työkaluja. Pääpaino auttaa lukijaa ymmärtämään mistä luonnollisen kielen käsittely koostuu kuten esikäsittely, kielianalyysimenetelmät ja oppimismenetelmät. Luvun esitetyt menetelmät ovat yleisiä, ja niitä käytetään myös kysymyksiin vastaamisessa. Kysymyksiin vastaamisen tiedonhakukomponentin kyky ymmärtää kysymyksen konteksti ja tarkoitus on olennaista. Erilaiset analyysikeinot auttavat järjestelmiä pääsemään tilanteeseen, jossa konteksti ja tarkoitus on mahdollisimman lähellä absoluuttista totuutta. Kysymyksen todellisen merkityksen ymmärtäminen voi olla ratkaisevaa oikean vastauksen löytämiseksi.

Kysymyksiin vastaamisen menetelmät luku 3 esittelee kysymyksiin vastaamista kuvaamalla konkreettisia metodeja, joita voidaan hyödyntää kysymyksiin vastaamisessa. Esimerkiksi sanaluokkien tunnistaminen auttaa ymmärtämään kysymyksiä, ja nimettyjen entiteettien tunnistaminen auttaa vastaamaan faktatietoa hakeviin kysymyksiin.

Entiteettien linkittäminen auttaa vähentämään monimerkityksellisyyttä sekä vahvistamaan kysymyksien ja vastauksien kontekstia. Lisäksi entiteettien linkittäminen auttaa järjestelmää keskittymään olennaiseen tiedonhakuprosessiin. Esimerkiksi, jos käyttäjä kysyy ”Kuka oli Urho Kekkonen?”, järjestelmän on tunnistettava ”Urho Kekkonen” entiteettinä ja haettava tietoa nimenomaan tähän liittyen.

Kysymyksiinvastaamisjärjestelmät (luku 4) ovat tutkielman ydin, ja ne kuvaavat erilaisia kysymyksiinvastaamisjärjestelmiä, käyden läpi niiden keskeisiä komponentteja. Kysymyksiinvastaamisjärjestelmien ytiminä voidaan toisaalta pitää niiden tiedonhakukomponentteja. Tutkimuksesta käy ilmi tehokkaan tiedonhaun tärkeys kysymyksiinvastaamisjärjestelmien luomista varten. Kysymyksen perusteellinen analysointi mahdollistaa tiedonhakujärjestelmän hakemaan juuri kysymykseen vastaavaa tietoa, eikä aihetta sivuavaa tietoa. Kysymysten tarkka analysointi ennen tiedonhakua on kriittinen vaihe kysymyksiinvastaamisjärjestelmissä. Analysointivaiheessa tunnistetaan kysymyksen avainsanat, semanttinen rakenne, entiteetit ja mahdolliset viittaukset, jotka auttavat tarkentamaan tiedonhakua. Ilman tätä perusteellista analysointia tiedonhakujärjestelmä voi palauttaa epätarkkoja tai epäolennaisia tuloksia, jotka eivät vastaa käyttäjän tarpeisiin.

Suomen kielen sovittaminen luku 5 antaa ymmärryksen siitä, että luonnollisen kielen käsittely koostuu samoista komponenteista eri kielillä, mutta vaativat oman räätälöinnin. Suomen kielen tekstiä esikäsitellessä on suositeltavaa ottaa huomioon kielen erityispiirteet, kuten rikas morfologia ja erilaiset taivutusmuodot, jotta esimerkiksi semanttinen analyysi olisi mahdollisimman tarkka. Parhaimmassa tapauksessa luonnollisen kielen käsittelyn järjestelmät ovat monikielisiä, mutta kielten ominaispiirteet tekevät siitä hankalaa. Suomen kielen sovittaminen luonnollisen kielen metodeihin tuo ongelmia juuri rikkaan morfologian, hajanaisen datan ja yleisen kielellisen kulttuuriin vuoksi. Vaikka kielen sovittaminen luonnollisen kielen käsittelyyn on haastavampaa, ratkaisuja on kuitenkin saatavilla. Turku NLP -tutkijoiden ryhmä on tehnyt edistyksellisiä työkaluja suomen kielen käsittelylle. Valmis ratkaisu morfologiseen analysointiin on luvussa 5 mainittu Omorfi. Hajanaisesta datasta riippumatta suomen kielelle on luotu muun muassa BERT-malli, FinGPT sekä korkean tarkkuuden nimettyjen entiteettien tunnistaja (Hämäläinen & Alnajjar, 2021). Tutkielman laajuus keskittyi enemmän klassisten menetelmien tutkimiseen, joten BERT-mallit ja transformerit eivät siihen kuulu. Kuitenkin on maininnan arvoista, että suomen kielelle on kehitetty klassisista menetelmistä edistyksellisempiä koneoppimista hyödyntäviä ratkaisuja.

6.1 Kysymyksiin vastaaminen usein kysytyissä kysymyksissä

Tämä aliluku arvioi usein kysyttyjen kysymysten asemaa kysymyksiin vastaamisessa. Aliluvussa otetaan huomioon tutkielman aikana ilmenneitä huomioita sekä yleistä pohdintaa usein kysyttyjen kysymysten luonteesta. Usein kysyttyjen kysymyksien tarjoaminen on toteutettava nopeasti sekä tehokkaasti. Käyttäjät haluavat löytää tiedon pikaisesti, ja siksi he turvautuvat usein kysyttyjen kysymysten sivun puoleen. Kysymykset ovat riippuvaisia verkkopalvelun kontekstista, ja muun muassa tämän vuoksi kysymysten analysoiminen on tärkeää. Kontekstin tietäminen etukäteen voi auttaa esimerkiksi monimerkityksellisyyden vähentämisessä, kun luodaan kysymyksiinvastaamisjärjestelmiä. Käyttäjät voivat muotoilla samaa tarkoittavia kysymyksiä eri tavalla, joten kysymysten analysoinnin pääpaino on suuri. Jos palvelu hyödyntää käyttäjäprofiileja, niin käyttäjäprofiilin tietoja, esimerkiksi ikää, voidaan käyttää personoimaan vastauksen muotoilutyyli.

Sääntöpohjainen kysymysten analysointi on erityisen tärkeää synonyymien tunnistamisessa ja semanttisessa ymmärryksessä. Vaikka se voi tehokkaasti kartoittaa eri muodossa esiintyviä kysymyksiä olemassa oleviin tietopankin vastauksiin, sääntöpohjaiset järjestelmät saattavat kohdata ongelmia, kun käsitellään suomen kielen monimutkaisuuksia. Esimerkiksi suomen kielessä sanoilla ”koti” ja ”asunto” on erilaiset konnotaatiot, vaikka ne molemmat viittaavat asumiseen. Konnotaatiolla tarkoitetaan tässä yhteydessä sanojen erilaista emotionaalista tai kulttuurista merkitystä. Jos järjestelmä virheellisesti kohtelee näitä termejä synonyymeinä, se saattaa tarjota epäolennaisia tai harhaanjohtavia vastauksia.

Koska usein kysytyt kysymykset ovat luonteeltaan yleisiä ja toistuvia, on tärkeää tarjota vastaukset riittävällä tarkkuudella ja nopeudella. Nopeuden varmistamiseksi voi olla hyödyllistä toteuttaa indeksointi- sekä välimuistiratkaisuja. Esimerkiksi tietyt kysymykset toistuvat useammin kuin toiset, ja niiden asettaminen etusijalle tärkeysjärjestyksessä voi olla hyödyllistä. Lisäksi esikäsittely aliluvun 2.1 ratkaisut ovat huomioitava olemassa oleviin resursseihin, jotta kysymyksiinvastaamisjärjestelmiä voidaan tarjota suurelle käyttäjämäärälle.

Tavanomaisiin kysymysten vastaamisjärjestelmiin verrattuna usein kysyttyjen kysymysten kontekstissa on lisäkehitystä. Esimerkiksi ennalta määrättyjen vastausten integroiminen järjestelmään, sekä potentiaalisten hyperlinkki-ohjauksien lisääminen lisätietoja varten. Lisäksi verkkopalvelun sisältäessä paljon päivittyvää tietoa, järjestelmän pitää

pysyä ajan tasalla tuoreimmista muutoksista. Kyseisessä tilanteessa on tärkeää ottaa huomioon välimuistiratkaisujen reagointikyky.

Usein kysyttyjen kysymysten virheidensietokyky voi olla pieni, riippuen sen käyttökontekstista. Parhaimmillaan järjestelmä pystyy tarjoamaan nopean vastauksen, joka on oikea tai tarjoaa tiedolle oikean polun. Pahimmillaan järjestelmä antaa virheellisen tai vanhan tiedon, jota noudattamalla käyttäjälle voi aiheutua harmia. Järjestelmän hiominen testausvaiheessa, sekä palautteen kerääminen ei-kriittisellä tiedolla voi olla hyödyksi mahdollisen harmin minimoimiseksi.

Kysymyksiinvastausjärjestelmien räätälöiminen omaan käyttötarkoitukseen voi tarjota pitkäkestoisemman ratkaisun, kuin yleisen järjestelmän käyttäminen. Muutoin tavanomainen prosessin pääpiirteet mahdollistavat kysymyksiin vastaamisen usein kysyttyjen kysymysten kontekstissa.

Usein kysyttyjen kysymysten prosessi:

1. **Kysymysten esikäsittely**: Kysymys pitää ensin esimerkiksi tokenisoida, stemmata ja tunnistaa sanaluokat. Jo kysymyksen esiprosessoinnista asti valinnat vaikuttavat tiedonhaun tehokkuuteen. Tiedonhakeminen jäsennetyllä kysymyksellä, antaa suuremman todennäköisyyden löytää kysymykseen olennaista tietoa.

2. **Kysymyksen analysointi**: Kysymyksen analysointi auttaa ymmärtämään kysymyksen todellista merkitystä ja parantaa tiedonhakua.

3. **Tiedonhaku**: Tiedonhakua voidaan tehdä klassisten metodien mukaisesti, kuten Boolen-, vektoriavaruusmallin tai todennäköisyyspohjaisen mallin avulla.

4. **Optimointi**: Tiedonhaun tehokkuuden säätäminen on erityisen tärkeää, jos tiedonhaku on resurssi-intensiivistä.

5. **Tiedon löytäminen ja vastauksen generointi**: Kun olennainen tieto on löydetty esimerkiksi käänteisestä indeksistä tai relaatiotietokannasta, on vastauksen tuottaminen seuraava askel.

6. **Vastauksen muotoilu**: Kohderyhmän tunteminen auttaa tarjoamaan sopivan vastausformaatin. Esimerkiksi nuoremmalle yleisölle kannattaa käyttää yksinkertaista sanastoa, kun taas teknisesti orientoituneelle yleisölle voi tarjota yksityiskohtaisempia vastauksia. Verkkopalvelun konteksti ja ontologia auttavat tarjoamaan kohderyhmälle sopivan vastausformaatin.

6.2 Suositukset ja tulevaisuuden suuntaviivat

Aluksi ajattelin, että luonnollisen kielen käsittely on suhteellisen rajattu tutkimusalue, mutta työskennellessäni aiheen parissa huomasin sen olevan huomattavasti laajempi ja monimuotoisempi kuin alun perin oletin. Klassiset menetelmät, kuten tekstinhaku ja -analyysi, ovat edelleen arvokkaita ja soveltuvia erityisesti kysymyksiinvastaamis-järjestelmille. Koneoppimiseen perustuvat menetelmät ovat esillä ja tuottavat lupaavia tuloksia, mutta klassiset menetelmät tarjoavat yhä oleellisen lähestymistavan, erityisesti kun otetaan huomioon suomen kielen erityispiirteet.

Jatkotutkimuksena voisi ottaa huomioon uudempien tekniikoiden käytön, kuten koneoppimistekniikat, kokeelliset vaiheet ja tulosten arvioinnin, jotka kaikki auttavat ymmärtämään menetelmien välisiä eroja. Erityistä huomiota olisi hyvä kiinnittää eettisiin kysymyksiin, kuten puolueettomuuteen, yksityisyyteen ja avoimuuteen, sillä ne ovat yhä tärkeämpiä tekijöitä järjestelmien kehityksessä.

Puhutun kielen analysointi avaa uusia mahdollisuuksia ja haasteita, kuten täytesanojen ja sanontojen tunnistamisen. Tämä on erityisen olennaista, sillä puhuttu kieli poikkeaa merkittävästi kirjakielestä. Sovelluksia suunniteltaessa on otettava huomioon myös käyttäjäpalaute ja eri tieteenalojen erityisvaatimukset, jotka voivat rajoittaa luonnollisen kielen käsittelyn yleispätevyyttä.

Lisäksi on syytä ottaa huomioon, että luonnollisen kielen käsittelyn sovellukset ovat usein erikoistuneita tiettyihin teollisuudenalojen tai akateemisten alojen konteksteihin. Tämä rajallinen konteksti voi tuoda lisähaasteita, sillä monet järjestelmät on suunniteltu yleiskäyttöön. Järjestelmän rajaamisen tarkempi tutkiminen voi olla hyödyllistä tulevaisuuden tutkimuksia varten, sillä personoidut järjestelmät voivat lisätä käyttäjätyytyväisyyttä.

Luvussa 4 mainittiin dialogijärjestelmät, mutta tarkempi läpikäynti ei kuulunut tutkielman laajuuteen. Dialogijärjestelmät ja chatbotit ovat kuitenkin alueita, jotka voivat tarjota uusia näkökulmia ja sovelluskohteita luonnollisen kielen käsittelylle.

Viitteet

Allam, A. M. N. & Haggag, M. H. (2012). The question answering systems: A survey. *International Journal of Research and Reviews in Information Sciences*, *2*(3).

Apache. (2023). *Lucene.* Haettu 10.11.2023 osoitteesta `https://lucene.apache.org/`.

Bocklisch, T., Faulkner, J., Pawlowski, N. & Nichol, A. (2017). Rasa: Open source language understanding and dialogue management. *arXiv:1712.05181 [cs.CL]*. doi: 10.48550/arXiv.1712.0518

Brunni, S., Lehto, L.-M., Jantunen, J. H. & Airaksinen, V. (2015). How to annotate morphologically rich learner language. principles, problems and solutions. *Bergen Language and Linguistics Studies*, *6*.

Cambridge Dictionary. (2023). *Entity.* Haettu 5.11.2023 osoitteesta `https://dictionary.cambridge.org/dictionary/english/entity`.

DBpedia. (2023). *Dbpedia.* Haettu 10.11.2023 osoitteesta `https://www.dbpedia.org/`.

Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv:1810.04805 [cs.CL]*. doi: 10.48550/arXiv.1810.04805

Ell, B. (2017). *User Interfaces to the Web of Data based on Natural Language Generation*. KIT Scientific Publishing.

Engati. (2023). *N-gram.* Haettu 2.11.2023 osoitteesta `https://www.engati.com/glossary/n-gram`.

Etzioni, O., Banko, M., Soderland, S. & Weld, D. S. (2008). Open information extraction from the web. *Communications of the ACM*, *51*(12), 68–74.

Fellbaum, Christiane. (2005). *Wordnet.* Haettu 10.11.2023 osoitteesta `https://wordnet.princeton.edu/`.

Ferrucci, D., Brown, E., Chu-Carroll, J., Fan, J., Gondek, D., Kalyanpur, A., . . . Welty, C. (2010). Building Watson: An Overview of the DeepQA Project. *AI Magazine*, *31*(3). doi: 10.1609/AIMAG.V31I3.2303

High, R. (2012). *The era of cognitive systems: An inside look at IBM Watson and how it works* (osa 1). IBM Redbooks.

Hirschman, L. & Gaizauskas, R. (2001). Natural language question answering: the view from here. *Natural Language Engineering*, *7*(4), 275–300.

Hyönä, J., Bertram, R. & Pollatsek, A. (2004). Are long compound words identified serially via their constituents? Evidence from an eyemovement-contingent display change study. *Memory & cognition*, *32*, 523–532.

Hämäläinen, M. (2018). Extracting a semantic database with syntactic relations for Finnish to boost resources for endangered uralic languages. *The Proceedings of Logic and Engineering of Natural Language Semantics 15*.

Hämäläinen, M. & Alnajjar, K. (2021). The current state of Finnish NLP. *arXiv:2109.11326 [cs.CL]*. doi: 10.48550/arXiv.2109.11326

Hämäläinen, M. & Rueter, J. (2018). Development of an open source natural language generation tool for Finnish. Teoksessa *Proceedings of the Fourth International Workshop on Computational Linguistics of Uralic Languages* (s. 51–58).

Indurkhya, N. & Damerau, F. J. (2010). *Handbook of natural language processing*. Chapman & Hall/CRC.

Jackson, P. & Moulinier, I. (2002). *Natural language processing for online applications: text retrieval, extraction and categorization*. John Benjamins Pub.

Jijkoun, V. & de Rijke, M. (2005). Retrieving answers from frequently asked questions pages on the web. Teoksessa *Proceedings of the 14th ACM international conference on Information and knowledge management* (s. 76–83).

Karlsson, F. (1990). Constraint grammar as a framework for parsing running text. Teoksessa *COLING 1990 Volume 3: Papers presented to the 13th International Conference on Computational Linguistics* (s. 168–173).

Keras. (2023). *Keras.* Haettu 10.11.2023 osoitteesta `https://keras.io/`.

Khyani, D., Siddhartha, B., Niveditha, N. & Divya, B. (2021). An interpretation of lemmatization and stemming in natural language processing. *Journal of University of Shanghai for Science and Technology*, *22*(10), 350–357.

Kokash, N. (2005). An introduction to heuristic algorithms. *Department of Informatics and Telecommunications, University of Trento*, 1–8.

Korenius, T., Laurikkala, J., Järvelin, K. & Juhola, M. (2004). Stemming and lemmatization in the clustering of Finnish text documents. Teoksessa *Proceedings of the thirteenth ACM international conference on Information and knowledge management* (s. 625–633).

Liddy, E. D. (2001). Natural language processing. Teoksessa *Encyclopedia of library and information science* (2nd ed. p., s. 2126–2136). Marcel Decker, Inc.

Lounela, M. (2006). *Korpukset ja kielen tutkiminen.* Haettu 10.11.2023 osoitteesta `https://www.kotus.fi/nyt/kolumnit_artikkelit_ja_esitelmat/kielipakina_%282006_2009%29/korpukset_ja_kielen_tutkiminen`.

Lourentzou, I., Manghnani, K. & Zhai, C. (2019). Adapting sequence to sequence

models for text normalization in social media. Teoksessa *Proceedings of the international AAAI conference on web and social media* (osa 13, s. 335–345).

Luoma, J., Oinonen, M., Pyykönen, M., Laippala, V. & Pyysalo, S. (2020). A Broad-coverage Corpus for Finnish Named Entity Recognition. Teoksessa *Proceedings of the Twelfth Language Resources and Evaluation Conference* (s. 4615–4624). European Language Resources Association.

Luotolahti, J., Kanerva, J., Laippala, V., Pyysalo, S. & Ginter, F. (2015). Towards universal web parsebanks. Teoksessa *Proceedings of the third international conference on Dependency Linguistics* (s. 211–220).

Manning, C. D. (2009). *An introduction to information retrieval*. Cambridge university press.

Marcu, D. (1997). The rhetorical parsing of unrestricted natural language texts. Teoksessa *35th Annual Meeting of the Association for Computational Linguistics and 8th Conference of the European Chapter of the Association for Computational Linguistics* (s. 96–103).

Martin, J. H. (2023). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Pearson Education.

McCallum, A. & Li, W. (2003). Early results for named entity recognition with conditional random fields, feature induction and web-enhanced lexicons. Teoksessa *Proceedings of the seventh conference on natural language learning at HLT-NAACL 2003* (s. 188–191). Association for Computational Linguistics.

Mendelson, E. (2009). *Introduction to mathematical logic*. CRC press.

Mishra, R., Bian, J., Fiszman, M., Weir, C. R., Jonnalagadda, S., Mostafa, J. & Del Fiol, G. (2014). Text summarization in the biomedical domain: a systematic review of recent research. *Journal of biomedical informatics*, *52*, 457–467.

Mohri, M. (1997). Finite-state transducers in language and speech processing. *Computational linguistics*, *23*(2), 269–311.

Murre. (2023). *Murre.* Haettu 10.11.2023 osoitteesta `https://pypi.org/project/murre/`.

Murthy, K., Padmanabhan, D. & Deshpande, P. M. (2012). Improving recall of regular expressions for information extraction. Teoksessa *Web Information Systems Engineering-WISE 2012: 13th International Conference, Paphos, Cyprus, November 28-30, 2012. Proceedings 13* (s. 455–467). Springer.

Partanen, N., Hämäläinen, M. & Alnajjar, K. (2019). Dialect text normalization to normative standard Finnish. Teoksessa *The fifth workshop on noisy user-generated text* (s. 141–146).

Peskoff, D. & Stewart, B. M. (2023). Credible without credit: Domain experts assess generative language models. Teoksessa *Proceedings of the 61st Annual Meeting*

of the Association for Computational Linguistics (Volume 2: Short Papers) (s. 427–438).

Plisson, J., Lavrac, N. & Mladenic, D. (2004). A rule based approach to word lemmatization. Teoksessa *Proceedings of IS* (osa 3, s. 83–86).

Rasa. (2023). *Rasahq.* Haettu 10.11.2023 osoitteesta https://github.com/RasaHQ/.

Reiter, E. & Dale, R. (1997). Building applied natural language generation systems. *Natural Language Engineering*, *3*(1), 57–87.

Reno, G. (2015). *10 Common Data Quality Issues (And How to Solve Them).* Haettu 2.11.2023 osoitteesta https://firsteigen.com/blog/10-common-data-quality-issues-and-how-to-solve-them/.

Saini, J. R. & Rakholia, R. M. (2016). On continent and script-wise divisions-based statistical measures for stop-words lists of international languages. *Procedia Computer Science*, *89*, 313–319.

scikit-learn. (2023). *Scikit-learn.* Haettu 10.11.2023 osoitteesta https://scikit-learn.org/stable/.

Sharma, R. K. & Joshi, M. (2020). An analytical study and review of open source chatbot framework, rasa. *International Journal of Engineering Research & Innovation*, *9*(06), 1011–1014.

SpaCy. (2023). *Spacy.* Haettu 10.11.2023 osoitteesta https://spacy.io/.

Tieteen termipankki. (2015). *Korpus.* Haettu 1.11.2023 osoitteesta https://tieteentermipankki.fi/wiki/Kielitiede:korpus.

Tieteen termipankki. (2021). *Diskurssi.* Haettu 2.11.2023 osoitteesta https://tieteentermipankki.fi/wiki/Ihmistieteet:diskurssi.

Tieteen termipankki. (2022). *Predikaattilogiikka.* Haettu 9.11.2023 osoitteesta https://tieteentermipankki.fi/wiki/Filosofia:predikaattilogiikka.

Tieteen termipankki. (2023). *Luonnollinen kieli.* Haettu 2.11.2023 osoitteesta https://tieteentermipankki.fi/wiki/Kielitiede:luonnollinen_kieli.

Turku NLP. (2020). *Finnish ner.* Haettu 9.11.2023 osoitteesta https://turkunlp.org/fin-ner.html.

Ursin, M. (2002). Triphone clustering in Finnish continuous speech recognition. *Diplomityö, Teknillinen korkeakoulu*.

Vijayarani, S., Ilamathi, M. J. & Nithya, M. (2015). Preprocessing techniques for text mining-an overview. *International Journal of Computer Science & Communication Networks*, *5*(1), 7–16.

Virtanen, A., Kanerva, J., Ilo, R., Luoma, J., Luotolahti, J., Salakoski, T., . . . Pyysalo, S. (2019). Multilingual is not enough: BERT for Finnish. *arXiv:1912.07076 [cs.CL]*. doi: 10.48550/arXiv.1912.07076

Voikko. (2023). *Voikko*. Haettu 10.11.2023 osoitteesta `https://voikko.puimula.org/`.

Wermter, S., Riloff, E. & Scheler, G. (1996). *Connectionist, Statistical and Symbolic Approaches to Learning for Natural Language Processing* (1st ed. 1996. p.). Springer.

Wilcock, G. (2001). Pipelines, templates and transformations: XML for natural language generation. Teoksessa *Proceedings of the 1st NLP and XML Workshop* (s. 1–8).

Yates, A., Banko, M., Broadhead, M., Cafarella, M. J., Etzioni, O. & Soderland, S. (2007). Textrunner: open information extraction on the web. Teoksessa *Proceedings of Human Language Technologies: The Annual Conference of the North American Chapter of the Association for Computational Linguistics* (s. 25–26).

Yule, G. (2022). *The study of language*. Cambridge university press.